

Multiomic GNN Techniques

A methods companion to the *Multiomic GNN Reading List* — architectures, mathematics,
and the graphs they run on

Version 1, 10 September 2026

Contents

How to use this companion	3
Notation	4
Part A — The graph is a modelling choice	5
A.1 What is a node?	5
A.2 Building the graph	6
A.3 Normalisation: the propagation matrix	9
A.4 Diagnostics before training	9
Part B — The machinery	11
B.1 Message passing	11
B.2 Where the GCN comes from	12
B.3 Aggregators and what they can distinguish	13
B.4 Attention	13
B.5 Heterogeneous and relational graphs	15
B.6 Depth: over-smoothing and over-squashing	15
B.7 Scalability	17
B.8 Pooling and graph-level readout	18
B.9 Self-supervision	18
B.10 Explainability	19
B.11 Beyond pairwise: hypergraphs, higher-order structure, and time	20
Part C — Backbones and objectives	22
C.1 Modelling counts properly	22
C.2 The variational autoencoder backbone	23
C.3 Graph autoencoders	24
C.4 Aligning distributions	24
C.5 Clustering heads	25
Part D — The applied architectures	27
D.1 The cell-graph template	27
D.2 Guidance graphs: integration without shared features	29
D.3 Spatial GNNs	30
D.4 Spatial multi-omics	33
D.5 Cell–cell communication	34
D.6 Gene regulatory networks	34
D.7 Patient-level multi-omics	36
D.8 Where foundation models fit	37
Part E — Evaluating any of this	38
E.1 Metrics, and what they actually measure	38
E.2 The ablations that decide whether a paper is about graphs	38
E.3 Leakage, in the forms it takes here	39

E.4 A training recipe that usually works	39
E.5 Software worth standardising on	40

Part F — The same ideas, in code **41**

F.1 Building the graph	41
F.2 The diagnostics of §A.4	42
F.3 A message-passing layer from scratch	42
F.4 The count likelihood	44
F.5 Propagation without parameters	44
F.6 A training loop with the checks built in	45
F.7 The ablation harness	45
F.8 Reading a benchmark table	46
F.9 Ten things worth carrying away	46

Appendix **48**

A. Complexity at a glance	48
B. Identities used above	48
C. Companion $\leftrightarrow$ reading list index	48
D. Figure credits	49

How to use this companion

The *Multiomic GNN Reading List* says what 241 papers claim and where they sit relative to one another. It deliberately does not stop to explain how the methods work. This companion does that job: it is the machinery behind the bibliography, written so that a reader can go from a section of the reading list to the equations that section's papers are built from, and back again.

Three commitments shape it.

Every architecture is drawn and written. Each family gets a schematic — what enters, what is learned, what comes out — and the equations that make the schematic precise, with tensor shapes annotated so the diagram and the algebra agree.

Nothing is asserted that can be computed. Where a claim about behaviour can be measured — over-smoothing, for instance — it is measured on a synthetic graph and the result is plotted rather than described (Figure 5).

The graph itself is treated as the model. Most of the published variation across these 241 papers is not in the message-passing layer, which is nearly always a GCN or a GAT; it is in how the graph was built and what the objective asks of it. Part A therefore comes before the neural network, not after it.

Cross-references. A marker like *[RL §4.1]* points to a section of the reading list; named methods appear there with citation counts and DOIs. Method names in this companion are given without citations for that reason.

Prerequisites. Linear algebra, probability at the level of exponential families, and enough deep learning to read an ELBO. Single-cell background is assumed at the level of: you know what a count matrix, a batch effect, and a UMAP are.

Notation

Symbol	Meaning
n, g, d	number of cells (or spots, patients), number of features, latent dimension
$X \in \mathbb{R}^{n \times g}$	feature matrix; x_c is the row for cell c , x_{cg} a single count
$\mathcal{G} = (\mathcal{V}, \mathcal{E})$	graph; $\mathcal{N}(v)$ the neighbours of v ; $d_v = \mathcal{N}(v) $
$A \in \{0, 1\}^{n \times n}$	adjacency matrix; W_{ij} a weighted variant
$\tilde{A} = A + I$	adjacency with self-loops; $\tilde{D}$ its degree matrix
$\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$	symmetrically normalised adjacency (“the propagation matrix”)
$L = I - \hat{A}$	normalised graph Laplacian
$H^{(\ell)} \in \mathbb{R}^{n \times d_\ell}$	node representations at layer ℓ ; $h_v^{(\ell)}$ one row
$W^{(\ell)}$	learnable weight matrix at layer ℓ (never a graph weight — those are W_{ij})
α_{vu}	attention coefficient on edge $u \rightarrow v$
z_c, u_c	latent embedding of cell c ; v_j the embedding of feature j
$\sigma(\cdot)$	logistic sigmoid; ReLU, ELU named explicitly
ℓ_c	library size of cell c
$\rho_{cg}, \theta_g, \pi_{cg}$	ZINB mean proportion, dispersion, zero-inflation
$\odot, \parallel$	elementwise product, concatenation

Shapes are given in the form $\mathbb{R}^{n \times d}$ whenever a matrix first appears in an equation. Where a paper’s own notation differs from this table, the table wins, and the difference is flagged.

Part A — The graph is a modelling choice

A.1 What is a node?

Single-cell biology has no graph in it. Nothing in a count matrix, a fragment file, or a Visium slide arrives with edges attached. Every graph in the 241 papers of the reading list was constructed by an author, and almost every disagreement between methods that looks architectural turns out, on inspection, to be a disagreement about that construction.

The first and most consequential decision is what a node represents. It fixes the size of the graph, the meaning of an edge, the task the model can express, and the evaluation that is even possible.

a One decision decides everything downstream: what is a node?

NODE	EDGE COMES FROM	WHAT THE MODEL DOES	READING LIST
 cell	kNN in a latent space	clustering, annotation, imputation, trajectories	§2
 cell + feature	measurement + prior biology	multi-omic integration, cross-modal translation	§3
 spot / cell in tissue	physical coordinates	domain identification, deconvolution, alignment, communication	§4-5
 gene	regulatory or PPI prior	link prediction (GRN), driver-gene node classification	§6, §7.2
 patient	sample-sample similarity per omic	subtyping, survival, drug response	§7.1, 7.3-7.4

the same GNN machinery runs on all five; what changes is the graph, and the graph is a modelling choice, not data

Figure 1: The five node types that organise the literature, and what each commits you to.

Read Figure 1 as a map of the reading list. Sections 2 through 5 are cell-and-spot graphs on the order of 10^4 – 10^6 nodes, where the task is to label or embed each node. Sections 6 and 7.2 are gene graphs of 10^4 nodes with strong priors and weak labels. Section 7.1 is patient graphs of 10^2 – 10^3 nodes, where the graph is small, the feature vector is enormous, and everything hinges on regularisation.

A useful discipline: before reading any method paper, state its node, its edge, its label, and its unit of evaluation. Most methodological confusion in this field dissolves at that point, and a surprising number of published comparisons turn out to be comparing models over different graphs rather than different architectures.

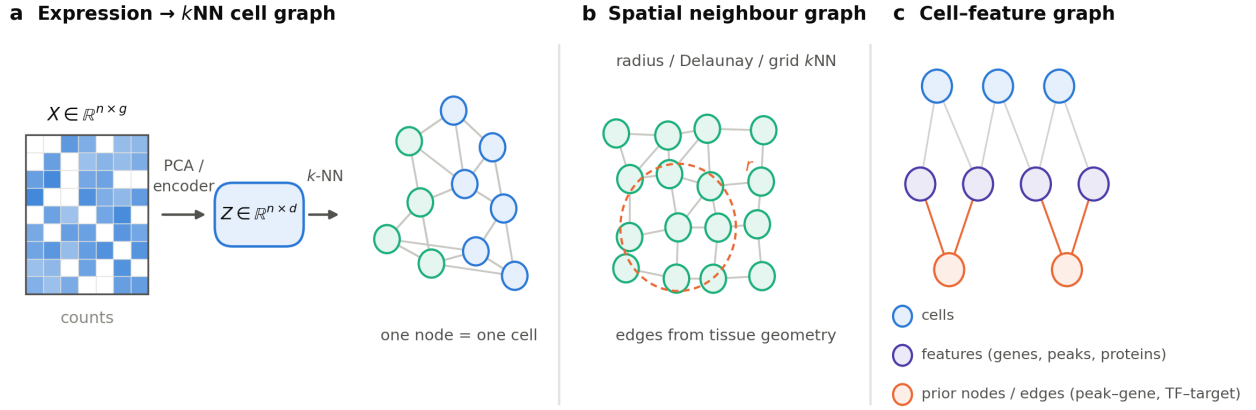


Figure 2: Three constructions cover nearly all of the literature: expression-derived k NN graphs, coordinate-derived spatial graphs, and bipartite cell–feature graphs carrying prior biology.

A.2 Building the graph

A.2.1 k -nearest-neighbour graphs on cells

The default construction, and the one inherited from Seurat and Scanpy, has four steps.

1. **Normalise and reduce.** Counts are library-size normalised, log1p-transformed, restricted to ~ 2000 highly variable genes, scaled, and projected to $d \approx 50$ principal components: $Z \in \mathbb{R}^{n \times d}$. This step, not the GNN, is where most of the denoising happens.
2. **Choose a metric.** Euclidean distance on PCs is standard; cosine distance is common for scATAC LSI components, where vector magnitude tracks depth rather than biology.
3. **Connect.** For each cell c , find its k nearest neighbours $\mathcal{N}_k(c)$ and set $A_{cu} = 1$ for $u \in \mathcal{N}_k(c)$.
4. **Symmetrise.** The k NN relation is not symmetric. Either take the union, $A \leftarrow \max(A, A^\top)$, which preserves connectivity for cells in sparse regions, or the intersection (mutual k NN), $A \leftarrow A \odot A^\top$, which removes hub artefacts at the cost of isolating rare cells. Union is the usual default; mutual k NN is worth trying whenever a UMAP shows suspicious bridges between clusters.

Weighting. A binary graph throws away the distances that produced it. Two standard repairs:

$$W_{cu} = \exp\left(-\frac{\|z_c - z_u\|^2}{2\sigma_c^2}\right), \quad \sigma_c = \|z_c - z_{(k)}\|, \quad (1)$$

with $z_{(k)}$ the k -th neighbour, giving an adaptive bandwidth that respects local density; or the fuzzy-simplicial weighting used by UMAP, which normalises each cell’s outgoing weights to a fixed “effective number of neighbours” before symmetrising with a probabilistic t-conorm, $W = P + P^\top - P \odot P^\top$. The second is what `scanpy.pp.neighbors` stores in `connectivities`, and it is what most single-cell GNN papers silently use when they say “the standard k NN graph”.

The choice of k . This is the single most important hyperparameter in the whole pipeline, and it is chronically under-reported. Small k (5–10) preserves rare populations and fine structure and pro-

duces a graph that fragments; large k (30–100) produces a smooth graph that merges neighbouring cell states. Because a GNN with L layers sees a k^L -sized neighbourhood, doubling k in a 2-layer model quadruples the effective receptive field: k and depth are not independent knobs, and a paper that tunes depth at fixed k has tuned only one thing badly.

Failure modes worth knowing. Batch effects put the strongest edges between cells of the same batch, so a graph built before integration encodes the batch structure the model is then asked to remove — this is why GLUE-style methods build the graph on features rather than cells, and why spatial methods, whose edges come from coordinates, are largely immune. Ambient RNA creates a false continuum that k NN faithfully reproduces. And in high dimension, distance concentration makes the ratio between nearest and farthest neighbour approach 1, which is the real argument for reducing to ~ 50 PCs before building anything.

A.2.2 Spatial graphs

When nodes are spots or segmented cells with coordinates $p_i \in \mathbb{R}^2$, edges come from geometry, and the model gains an inductive bias that is genuinely independent of the expression it is fitting. Four constructions dominate:

- **Radius (fixed- r) graphs.** $A_{ij} = \mathbb{1}\{\|p_i - p_j\|_2 \leq r\}$. Honest about physical scale, but degree varies with cell density, so dense tumour regions get hub nodes and sparse stroma gets isolates.
- **k NN on coordinates.** Fixed degree, variable physical scale. Preferable for imaging-based data (MERFISH, Xenium, CosMx) with strongly varying density.
- **Delaunay triangulation / Voronoi adjacency.** Scale-free and parameter-free, edges connect cells that actually touch. The natural choice for cell–cell communication models, where physical contact is the mechanism being modelled. Prune long edges at tissue borders, or the convex hull will connect cells across empty space.
- **Grid adjacency.** For Visium’s hexagonal lattice, the six immediate neighbours (or the 18 in a two-ring neighbourhood) — deterministic and reproducible.

Scale is the parameter that matters, not the construction. A radius chosen at the scale of a cell diameter models contact; one at 200 μm models a niche; one at 1 mm models an anatomical region. The right answer follows from the biological question, and a paper that reports ARI improvements without reporting r or k in microns has reported nothing reproducible.

Mixing expression into a spatial graph. Methods diverge sharply here, and the divergence is the real taxonomy of [RL §4.1]:

$$W_{ij} = \underbrace{\exp\left(-\frac{\|p_i - p_j\|^2}{2l^2}\right)}_{\text{geometry}} \cdot \underbrace{\exp\left(-\frac{\|x_i - x_j\|^2}{2\tau^2}\right)}_{\text{expression, optional}}. \quad (2)$$

SpaGCN folds histology into the first term by treating a colour-derived statistic as a third coordinate z , so that visually distinct neighbours are pushed apart before any learning happens. STAGATE leaves the graph binary and learns the second term as attention. Spatial-MGCN and other multi-view models keep the two graphs separate and fuse their outputs. These are three different answers

to one question — *should the graph encode expression similarity, or should the network learn it?* — and the empirical answer, across the benchmarks in [RL §9], is that learning it wins when domains have sharp boundaries and hurts when the signal is weak and the attention has nothing to latch onto.

A.2.3 Feature graphs and guidance graphs

Here nodes are genes, peaks, or proteins, and edges come from prior biology rather than from the data at hand:

- **Peak** → **gene**: a chromatin peak within the gene body or a promoter window (typically ± 2 kb, sometimes to 150 kb with distance decay). This is the edge set that makes RNA + ATAC integration possible without shared features, and it is Signac/ArchR output, not something a GNN discovers.
- **TF** → **target**: motif presence in an accessible region, or a curated regulatory database.
- **Protein–protein interaction**: STRING, BioGRID, Reactome; the substrate for driver-gene models [RL §7.2].
- **Gene–gene co-expression or pathway membership**: cheap, and the weakest of the four, because it is derived from the same data the model then fits.

Two properties distinguish these graphs from cell graphs and change how they must be handled. They are **signed and weighted** — a peak in a repressor’s binding site should carry a negative weight, and GLUE’s guidance graph does exactly this — and they are **fixed across cells**, which means one graph is shared by the entire dataset, so its size is $g \times g$ rather than $n \times n$ and its cost does not grow with the experiment.

The bipartite cell–feature graph in Figure 2c is the object that unifies [RL §3]: cells on one side, features on the other, observed measurements as edges, prior biology as edges among features. SIMBA takes this furthest by embedding cells, genes, peaks, and TF motifs as nodes in a single heterogeneous graph, so that “which genes mark this cell type” becomes a nearest-neighbour query in one shared space.

A.2.4 Patient-similarity networks

With $n \sim 200$ –1000 patients and $p \sim 20,000$ features per omic, the graph is small and the feature vector is not. The standard construction, from MOGONET onward:

$$s_{ij} = \frac{x_i^\top x_j}{\|x_i\| \|x_j\|}, \quad A_{ij} = \mathbb{1}\{s_{ij} > \epsilon\} \cdot s_{ij}, \quad (3)$$

with ϵ chosen so that the mean degree hits a target (often 5–10). One such graph is built per omic and the graphs are kept separate — fusion happens later (§D.7). Similarity Network Fusion (SNF) is the classical alternative, iteratively diffusing each network towards the others before any learning.

The honest difficulty here is statistical, not architectural. With a few hundred nodes and a two-layer GCN, the model has more parameters than patients, the graph is dense enough that two propagation steps reach most of the cohort, and cross-validation is being done on the same similarity matrix

that defines the neighbourhoods. Several independent re-evaluations in [RL §9] find that a regularised linear model on concatenated omics is competitive with most of [RL §7.1] under matched preprocessing. That is not a reason to skip the section; it is the control any new method there has to beat.

A.3 Normalisation: the propagation matrix

Given A , essentially every model in the reading list propagates with one of three operators:

$$\hat{A}_{\text{sym}} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}, \quad \hat{A}_{\text{rw}} = \tilde{D}^{-1} \tilde{A}, \quad \hat{A}_{\text{mean}} = D^{-1} A. \quad (4)$$

The symmetric form is the GCN default. Its eigenvalues lie in $(-1, 1]$, which keeps repeated multiplication stable; the self-loop in $\tilde{A} = A + I$ is what shifts the spectrum off -1 and stops two-layer propagation from oscillating on bipartite-ish structures. Its cost is that a node’s own signal is scaled by $1/\tilde{d}_v$, so high-degree cells are damped relative to low-degree ones — the reason hub artefacts in a union- k NN graph show up as unexpectedly smooth embeddings for exactly the cells you care least about smoothing.

The random-walk form gives every node a row that sums to one, making $\hat{A}h$ a genuine neighbourhood average, which is the right choice when degree varies by an order of magnitude across the graph — dense tumour cores versus sparse stroma, for instance. GraphSAGE’s mean aggregator is $\hat{A}_{\text{mean}}$ with the self term kept separate:

$$h_v^{(\ell)} = \sigma \left(W^{(\ell)} \left[h_v^{(\ell-1)} \parallel \frac{1}{d_v} \sum_{u \in \mathcal{N}(v)} h_u^{(\ell-1)} \right] \right), \quad (5)$$

and keeping the self term separate rather than folding it into a self-loop is what lets a GraphSAGE layer preserve a cell’s own identity while its neighbourhood is averaged — a small architectural detail that matters a great deal for rare-cell-type annotation.

A.4 Diagnostics before training

Four checks, none expensive, each of which has saved more time than it costs:

Degree distribution. Plot it. A long right tail means hubs, which after two layers dominate the representation of everything they touch. A spike at zero means isolated nodes, which a GCN will represent using only their self-loop and which will then cluster together as an artefactual “cluster”.

Edge purity against known labels. If any labels exist (cell types, spatial domains, tumour vs normal), compute the fraction of edges joining same-label nodes. This is the ceiling on what message passing can do for you: a graph with 60% edge purity is telling you that four in ten messages are contamination, and no attention mechanism recovers fully from that.

The permuted-graph control. Train the model on a degree-preserving rewiring of the graph. If performance barely drops, the graph is not carrying the signal — the encoder and the loss are —

and the paper you are reading (or writing) is not really about graphs. This single ablation, run consistently, would have deflated a noticeable fraction of the claims in [RL §2.1].

Connectivity. Count connected components. Isolated fragments cannot exchange information with the rest of the graph at any depth, which is usually a sign that k is too small or that a mutual- k NN intersection has been too aggressive.

Part B — The machinery

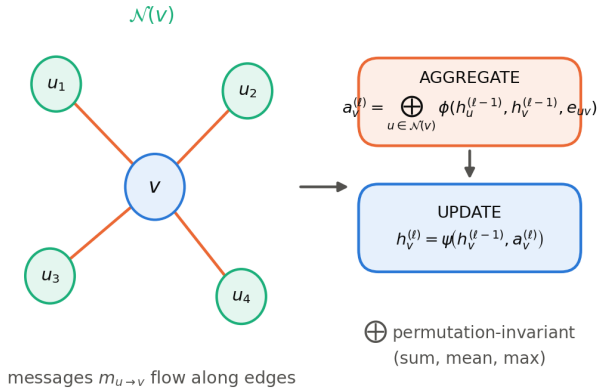
B.1 Message passing

Every architecture in the reading list, from a two-layer GCN on a k NN graph to a graph transformer on a tissue slice, is an instance of one template. A layer updates each node from its neighbours in two steps:

$$a_v^{(\ell)} = \bigoplus_{u \in \mathcal{N}(v)} \phi(h_u^{(\ell-1)}, h_v^{(\ell-1)}, e_{uv}), \quad h_v^{(\ell)} = \psi(h_v^{(\ell-1)}, a_v^{(\ell)}), \quad (6)$$

where ϕ is a message function, $\bigoplus$ a permutation-invariant aggregator, and ψ an update function. $h_v^{(0)} = x_v$, and after L layers $h_v^{(L)}$ summarises the L -hop neighbourhood of v .

a One round of message passing at node v



b Receptive field after L layers

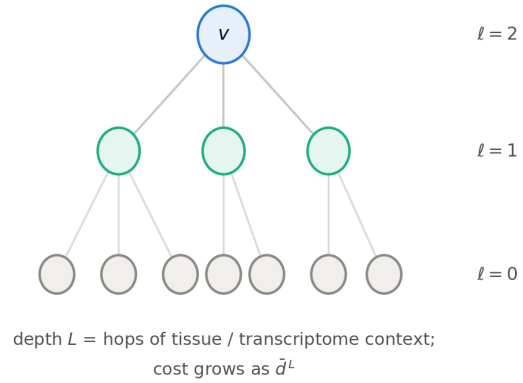


Figure 3: Message passing and the receptive field it induces. Each layer adds one hop; cost grows with the average degree raised to the depth.

Why permutation invariance is not optional. A graph has no canonical node order. If $\bigoplus$ were order-dependent, relabelling cells — reading the same AnnData object with a different sort — would change the model’s predictions. Sum, mean, and max qualify; concatenation does not. This is the entire reason GNN layers look the way they do, and it is the property that makes a GNN, rather than an MLP on a flattened adjacency row, the right object for this data.

Invariance versus equivariance. A message-passing layer is *equivariant*: permuting the nodes permutes the outputs identically, $f(PX, PAP^\top) = Pf(X, A)$. Node-level tasks — clustering, annotation, imputation, domain identification — want exactly this. Graph-level tasks — tumour-microenvironment classification, patient outcome from a tissue graph — need a final *invariant* read-out that collapses the node set into one vector (§B.8).

Matrix form and cost. For the common case where ϕ is linear and $\oplus$ is a weighted sum, the whole layer is one sparse matrix product:

$$H^{(\ell)} = \sigma(\hat{A}H^{(\ell-1)}W^{(\ell)}), \quad H^{(\ell)} \in \mathbb{R}^{n \times d_\ell}, \quad W^{(\ell)} \in \mathbb{R}^{d_{\ell-1} \times d_\ell}. \quad (7)$$

The cost is $O(|\mathcal{E}|d + nd^2)$ per layer in time and $O(nd)$ in memory for activations. With $n = 10^5$ cells, $k = 15$, and $d = 128$, that is roughly 1.5×10^6 edges and a trivially small dense term — GNNs on single-cell data are cheap until the graph gets dense or the model gets deep, which is why the scalability literature ([RL §II.6]) matters more for web-scale graphs than for most single-cell work. It becomes binding for Xenium-scale spatial data and for atlas integration at 10^7 cells.

Reading the receptive field biologically. L is not a free hyperparameter. On a k NN cell graph, $L = 2$ means “this cell’s state is described by its transcriptional neighbourhood and their neighbourhoods” — roughly, a cell type and its immediate continuum. On a spatial graph with r set at one cell diameter, $L = 2$ means “two cell layers”, which is a plausible scale for juxtacrine signalling and a poor one for a tissue domain. Choose L from the biology, then check that the model survives it (§B.6).

B.2 Where the GCN comes from

The graph convolution used by most of [RL §2] and [RL §4] is not a heuristic; it is a first-order truncation of a spectral filter, and the derivation is worth doing once because it explains both the self-loop and the $1/\sqrt{d_u d_v}$ that would otherwise look arbitrary.

Step 1 — the graph Fourier transform. The normalised Laplacian $L = I - D^{-1/2}AD^{-1/2}$ is real, symmetric, and positive semi-definite, so $L = U\Lambda U^\top$ with orthonormal eigenvectors U and eigenvalues $0 = \lambda_1 \leq \dots \leq \lambda_n \leq 2$. Define $\hat{x} = U^\top x$ as the graph Fourier transform. Small λ corresponds to eigenvectors that vary slowly across edges — the graph analogue of low frequency.

Step 2 — spectral filtering. A filter is a function $g_\theta(\Lambda)$ applied in the spectral domain: $y = U g_\theta(\Lambda) U^\top x$. This is fully general and useless in practice: the eigendecomposition is $O(n^3)$, the filter has n free parameters, and U is dense, so nothing is local.

Step 3 — polynomial filters. Restrict g_θ to a degree- K polynomial, $g_\theta(\Lambda) = \sum_{k=0}^K \theta_k \Lambda^k$. Then $U g_\theta(\Lambda) U^\top = \sum_k \theta_k L^k$, no eigendecomposition is needed, and because $(L^k)_{ij} = 0$ whenever the shortest path between i and j exceeds k , the filter is exactly K -localised. ChebNet uses Chebyshev polynomials T_k of the rescaled $\tilde{L} = \frac{2}{\lambda_{\max}} L - I$ for numerical stability, computed by the recurrence $T_k(\tilde{L}) = 2\tilde{L}T_{k-1} - T_{k-2}$.

Step 4 — the GCN simplification. Take $K = 1$ and approximate $\lambda_{\max} \approx 2$:

$$g_\theta * x \approx \theta_0 x + \theta_1 (L - I)x = \theta_0 x - \theta_1 D^{-1/2} A D^{-1/2} x.$$

Constrain $\theta = \theta_0 = -\theta_1$ to halve the parameters and control overfitting:

$$g_\theta * x \approx \theta (I + D^{-1/2} A D^{-1/2}) x.$$

The operator $I + D^{-1/2}AD^{-1/2}$ has eigenvalues in $[0, 2]$, so stacking it repeatedly amplifies whatever sits near 2 and the network becomes numerically unstable. The **renormalisation trick** fixes this by folding the identity into the adjacency before normalising, $I + D^{-1/2}AD^{-1/2} \rightarrow \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2} = \hat{A}$, which shifts the spectrum into $(-1, 1]$ and yields Equation (7).

Two consequences follow directly, and both are visible in practice. A GCN layer is a **low-pass filter**: it damps the high-frequency components of the signal, which is why it denoises, and why stacking it too many times leaves nothing but the lowest frequency (§B.6). And a GCN is **fixed**: the coefficient on edge (u, v) is $1/\sqrt{\tilde{d}_u\tilde{d}_v}$ regardless of what the two cells are expressing.

B.3 Aggregators and what they can distinguish

The choice of $\oplus$ determines what the layer can tell apart. The Weisfeiler–Leman (1-WL) colour-refinement test is the right yardstick: no message-passing GNN of the form (6) can distinguish two graphs that 1-WL cannot, and a GNN attains that bound only if both its aggregator and its update are injective on multisets.

- **Mean** loses multiset cardinality: $\{a, a, b\}$ and $\{a, b\}$ have the same mean. On a cell graph this means a layer cannot tell “surrounded by six T cells” from “surrounded by two T cells” once the mean is taken — fatal for niche characterisation, harmless for smoothing.
- **Max** keeps only the extreme: $\{a, b, b\}$ and $\{a, b\}$ collapse. Useful for detecting the presence of a signal, blind to its prevalence.
- **Sum** is injective on multisets of bounded size, which is why GIN uses it:

$$h_v^{(\ell)} = \text{MLP}^{(\ell)} \left((1 + \epsilon^{(\ell)}) h_v^{(\ell-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(\ell-1)} \right). \quad (8)$$

The $(1 + \epsilon)$ term keeps the node’s own representation distinguishable from its neighbours’ contributions, and the MLP (rather than a single linear layer plus nonlinearity) is what makes the update injective.

The practical translation. For spatial tissue graphs, where “how many of each cell type are around me” is the biological quantity of interest, sum-aggregation with degree as an explicit feature is the principled choice, and mean-aggregation is what most published methods actually use. For k NN cell graphs where degree is constant by construction, mean and sum differ only by a scale factor the next weight matrix absorbs, and the distinction is empty. Knowing which regime you are in prevents importing an argument from one into the other.

B.4 Attention

Attention replaces the fixed coefficient with a learned, data-dependent one.

For GAT, with $W \in \mathbb{R}^{d' \times d}$ and $a \in \mathbb{R}^{2d'}$:

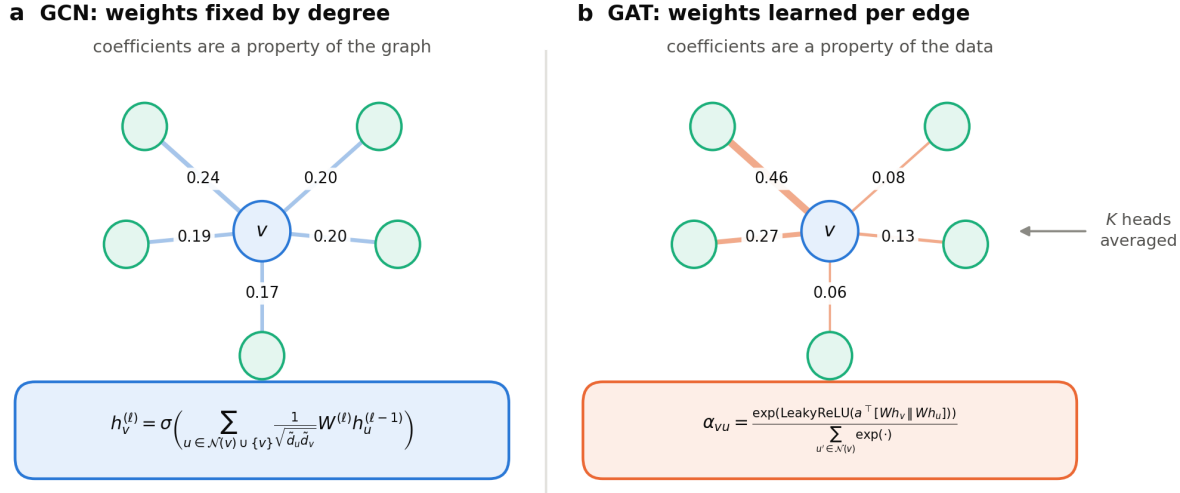


Figure 4: The same neighbourhood under a GCN and under a GAT. The coefficients are the entire difference.

$$e_{vu} = \text{LeakyReLU}(a^\top [Wh_v \parallel Wh_u]), \quad \alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{u' \in \mathcal{N}(v)} \exp(e_{vu'})}, \quad (9)$$

$$h'_v = \sigma \left(\sum_{u \in \mathcal{N}(v)} \alpha_{vu} Wh_u \right), \quad \text{or with } K \text{ heads } h'_v = \left\|_{k=1}^K \sigma \left(\sum_u \alpha_{vu}^k W^k h_u \right) \right\|.$$

Heads are concatenated in hidden layers and averaged in the output layer. Note that $\alpha_{vu} \neq \alpha_{uv}$ in general: attention is directed even on an undirected graph, and both directions are computed.

The static-attention problem. In (9), a splits as $[a_1 \parallel a_2]$ and $e_{vu} = \text{LeakyReLU}(a_1^\top Wh_v + a_2^\top Wh_u)$. The first term is constant within a node’s softmax, so the *ranking* of neighbours induced by attention is the same for every query node — GAT can scale a global ranking of neighbours, but not reorder it per node. GATv2 fixes this by applying the nonlinearity before the projection:

$$e_{vu} = a^\top \text{LeakyReLU}(W[h_v \parallel h_u]), \quad (10)$$

which is strictly more expressive at the same cost. Spatial methods that depend on attention to *suppress* a neighbour across a domain boundary — STAGATE and its descendants in [RL §4.1] — are exactly the setting where the distinction bites, and a GATv2 layer is usually a free improvement.

Graph transformers. Removing the graph from the attention denominator gives full self-attention: every node attends to every other, $O(n^2)$, with the graph reintroduced as a bias term (Graphormer’s spatial encoding adds a learned function of shortest-path distance to the attention logits) or as a positional encoding built from Laplacian eigenvectors $U_{:,2:m}$. For 10^5 cells the quadratic term is prohibitive, which is why the transformer methods in this literature — scMoFormer, Hist2ST, TCGN, DeepMAPS — apply attention within small neighbourhoods, within a patch, or over features rather than over all cells. Sign ambiguity in Laplacian eigenvectors (u and $-u$ are both valid) must be

handled by random sign flipping during training, or the encoding becomes noise.

What attention is not. Learned α_{vu} is routinely presented as an explanation — “the model attends to these neighbours, therefore they matter”. Attention weights are not identifiable: different weightings compose to nearly identical outputs, they shift under reinitialisation, and they are only loosely related to gradient-based attribution. Treat them as a hypothesis generator, and see §B.10.

B.5 Heterogeneous and relational graphs

A bipartite cell–feature graph has two node types and at least two edge types, so a single W is the wrong object. Relational GCN gives each relation $r \in \mathcal{R}$ its own transform:

$$h_v^{(\ell)} = \sigma \left(W_0^{(\ell)} h_v^{(\ell-1)} + \sum_{r \in \mathcal{R}} \sum_{u \in \mathcal{N}_r(v)} \frac{1}{c_{v,r}} W_r^{(\ell)} h_u^{(\ell-1)} \right), \quad (11)$$

with $c_{v,r} = |\mathcal{N}_r(v)|$. The parameter count grows linearly in $|\mathcal{R}|$, so R-GCN uses basis decomposition, $W_r = \sum_{b=1}^B a_{rb} V_b$, sharing $B \ll |\mathcal{R}|$ basis matrices across relations — the same trick that makes knowledge-graph models tractable, and the reason a heterogeneous single-cell graph with a dozen edge types does not blow up.

Heterogeneous graph transformers (HGT), used by DeepMAPS, go further: attention parameters are typed by the triple (source type, edge type, target type), so a cell←gene message and a gene←gene message are computed by different projections. For single-cell work the practical question is whether the extra typing earns its parameters on datasets of realistic size; the ablations in the source papers are usually the only evidence, and they are usually thin.

Metapaths are the cheaper alternative: define a path template such as cell→gene→cell, materialise the induced homogeneous graph (“cells sharing high expression of the same genes”), and run a standard GNN on it. This is what several [RL §7] methods do implicitly when they build a patient graph from a gene-level similarity.

B.6 Depth: over-smoothing and over-squashing

Deep GNNs do not work the way deep CNNs do, and the reason is visible in the propagation matrix.

The linear-algebra statement. Ignore nonlinearities and weights; propagation is $H^{(L)} = \hat{A}^L H^{(0)}$. Because $\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$ is symmetric, it has real eigenvalues $1 = \mu_1 > \mu_2 \geq \dots \geq \mu_n > -1$ on a connected non-bipartite graph. Expanding in the eigenbasis, $\hat{A}^L h = \sum_i \mu_i^L \langle h, u_i \rangle u_i$, every component except the first decays geometrically, and

$$\hat{A}^L H^{(0)} \longrightarrow u_1 u_1^\top H^{(0)}, \quad u_1 = \frac{\tilde{D}^{1/2} \mathbf{1}}{\|\tilde{D}^{1/2} \mathbf{1}\|}, \quad \text{i.e.} \quad h_v \rightarrow \sqrt{\tilde{d}_v} \frac{\sum_u \sqrt{\tilde{d}_u} h_u^{(0)}}{\sum_w \tilde{d}_w}. \quad (12)$$

Every node ends up pointing in the *same direction*, its length set only by its own degree: all cell-type

information is gone and only degree survives. (With the random-walk operator $\hat{A}_{\text{rw}}$ the limit is the cleaner $\mathbf{1}\pi^\top H^{(0)}$ with $\pi_v = \tilde{d}_v / \sum_u \tilde{d}_u$ — every node converging to the identical degree-weighted average.) The rate is governed by the spectral gap $1 - \mu_2$, which for a well-clustered graph is small (slow collapse) and for a well-mixed graph is large (fast collapse).

The measurable statement. Define the Dirichlet energy

$$E(H) = \frac{1}{2} \sum_{(u,v): \tilde{A}_{uv} > 0} \left\| \frac{h_u}{\sqrt{\tilde{d}_u}} - \frac{h_v}{\sqrt{\tilde{d}_v}} \right\|^2 = \text{tr}(H^\top L H), \quad (13)$$

the sum running over ordered pairs, which measures how much the representation varies across edges. $E \rightarrow 0$ is over-smoothing, made precise.

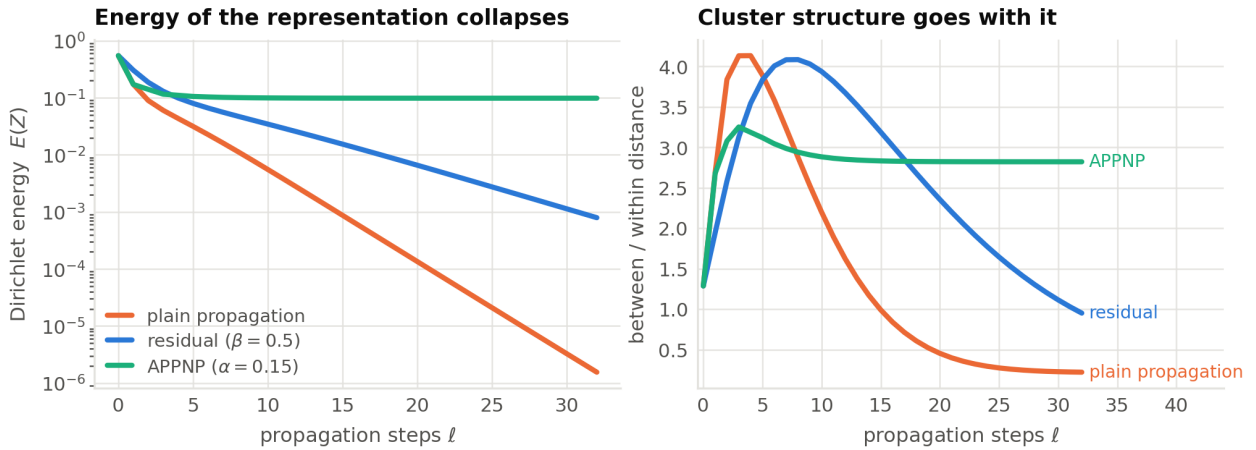


Figure 5: Over-smoothing measured on a three-community stochastic block model with 360 nodes. Left: Dirichlet energy, normalised by $\text{tr}(H^\top H)$, on a log scale. Right: between-cluster over within-cluster distance, the quantity clustering actually depends on.

Three things in Figure 5 are worth pausing on. First, plain propagation loses five orders of magnitude of energy in 30 steps — over-smoothing is not a subtle effect. Second, the separability curve *rises before it falls*: two to four propagation steps genuinely improve cluster structure (from 1.3 to 3.8 here), which is why 2-layer GCNs work and is the entire empirical case for graph methods on single-cell data. Third, the fix is not depth-agnostic regularisation but re-injection of the input: APPNP holds separability at 2.8 after 32 steps where plain propagation has fallen to 0.22.

Personalised PageRank propagation. The APPNP construction decouples depth from parameters. Predict once with an MLP, $H = f_\theta(X)$, then propagate with restart:

$$Z^{(0)} = H, \quad Z^{(t+1)} = (1 - \alpha)\hat{A}Z^{(t)} + \alpha H, \quad Z^{(\infty)} = \alpha \left(I - (1 - \alpha)\hat{A} \right)^{-1} H. \quad (14)$$

The limit is the personalised PageRank matrix applied to H , with teleport probability α (0.1–0.2 typical). Because no weights sit inside the propagation, $T = 10$ steps cost ten sparse products

and zero extra parameters, and the fixed point is a genuinely deep receptive field that does not collapse. For single-cell graphs this is the most under-used idea in the literature: it gives the long-range smoothing that spatial domain identification wants without the degeneration that motivates all the contrastive patches in [RL §4.1].

Other fixes, ranked by how often they help. Initial/residual connections, $h^{(\ell)} = (1-\beta)\hat{A}h^{(\ell-1)} + \beta h^{(0)}$ (GCNII adds an identity-mapping term to the weights as well); PairNorm and similar normalisations that hold total pairwise distance constant; DropEdge, which removes a random fraction of edges each epoch and slows convergence to the fixed point; and jumping-knowledge readouts that concatenate or max-pool $\{h^{(1)}, \dots, h^{(L)}\}$ so the model can select a per-node depth.

Over-squashing is the second, distinct failure. Information from an exponentially growing L -hop neighbourhood is compressed into a fixed-size vector, and when the graph has a bottleneck — a bridge between two dense communities — the Jacobian $\partial h_v^{(L)} / \partial x_u$ across that bridge decays exponentially in the number of hops. Nodes far apart in graph distance cannot influence each other however long you train. In practice this shows up in spatial graphs of elongated structures (a gland, a vessel, a cortical layer) where the relevant context is many hops along a thin path. Remedies: rewire with a few long-range edges, use a diffusion operator (14) rather than repeated local averaging, or add a virtual global node — cheap, effective, and rarely used in this literature.

B.7 Scalability

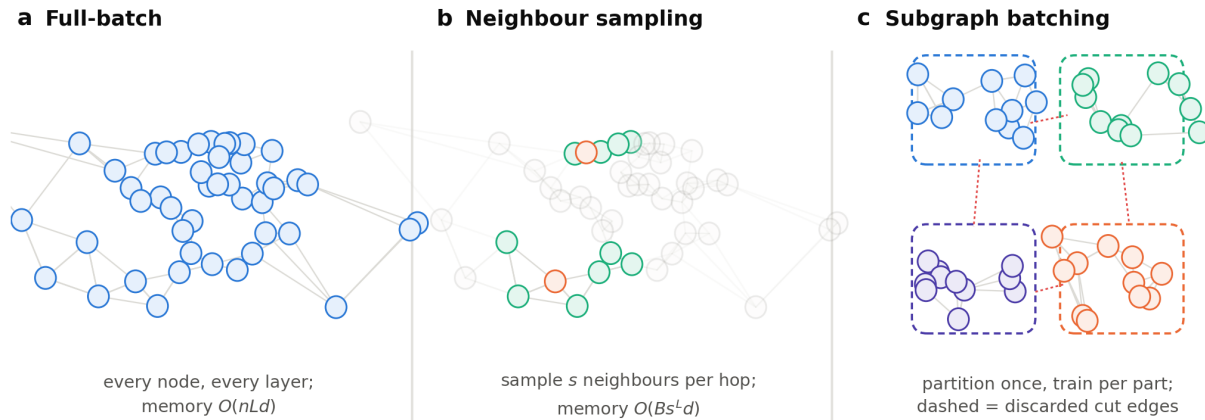


Figure 6: Three ways to train on a graph too large for full-batch gradients.

Full-batch. Compute $\hat{A}H^{(\ell)}W^{(\ell)}$ over all nodes; store all activations for the backward pass. Memory is $O(nLd)$: at $n = 10^6$, $L = 3$, $d = 256$ in float32 that is roughly 3 GB for activations alone before gradients and optimiser state. Fine for 10^5 cells on a 24 GB card, not for an atlas.

Neighbour sampling (GraphSAGE). Sample s neighbours per node per layer, giving a per-batch cost of $O(Bs^L)$ node evaluations for batch size B . With $s = 10$, $L = 2$, $B = 1024$ that is $\sim 10^5$ node evaluations — constant in n . The estimator is biased for nonlinear layers, and variance grows with depth, which is why s is usually taken larger in the first hop than in later ones.

Layer-wise / importance sampling (FastGCN). Sample a fixed number of nodes *per layer* rather

than per node, using an importance distribution $q(u) \propto \|\hat{A}_{:,u}\|^2$, which bounds the total sample count at $O(sL)$ regardless of degree.

Subgraph batching (Cluster-GCN, GraphSAINT). Partition the graph once with METIS into densely connected parts and train on whole parts. Within a part, message passing is exact; between parts, cut edges are dropped. Cluster-GCN mitigates this by sampling several parts per batch and restoring the edges between them. This is the most natural fit for single-cell data, because a k NN cell graph partitions almost perfectly along cell types — which is also the catch: the partitions correlate with the labels, so the gradient in each batch is systematically biased. Randomising the part composition per batch is not optional here.

Precomputation (SGC). Remove the nonlinearities entirely, $\hat{Y} = \text{softmax}(\hat{A}^K X W)$, precompute $\hat{A}^K X$ once, then train a linear model on the result. This turns a GNN into logistic regression on smoothed features, runs in seconds on millions of cells, and is *the* baseline every method in [RL §2.1] and [RL §7.1] should be compared against. It very often wins on ARI per unit of compute, and its absence from a benchmark table is informative.

B.8 Pooling and graph-level readout

Node-level tasks stop at $H^{(L)}$. Graph-level tasks — a tissue region classified as responder/non-responder, a patient graph mapped to a survival risk, a cell-neighbourhood graph mapped to a phenotype [RL §4.6] — need an invariant readout.

Global readouts. $h_{\mathcal{G}} = \sum_v h_v$, $\text{mean}_v h_v$, or $\max_v h_v$; sum preserves size information, mean discards it, and for tissue graphs the number of cells in a region is usually a real covariate, so mean-pooling silently removes a predictor.

Attention pooling learns which nodes count:

$$h_{\mathcal{G}} = \sum_v \gamma_v h_v, \quad \gamma_v = \text{softmax}_v(w^\top \tanh(V h_v)), \quad (15)$$

which is the standard multiple-instance-learning readout and the right default for tissue graphs where a small region drives the label.

Hierarchical pooling (DiffPool). Learn a soft assignment $S^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell+1}}$ with a GNN, then coarsen:

$$X^{(\ell+1)} = S^{(\ell)\top} H^{(\ell)}, \quad A^{(\ell+1)} = S^{(\ell)\top} A^{(\ell)} S^{(\ell)}. \quad (16)$$

Interpretable in principle — the clusters are supposed to be tissue motifs — and unstable in practice, needing entropy and link-prediction auxiliary losses to avoid degenerate assignments. Top- k pooling (SAGPool, gPool), which scores nodes and keeps the top fraction, is cheaper and usually as good.

B.9 Self-supervision

Labels are scarce in this field; graphs are not. Three families cover the reading list.

Deep Graph Infomax (DGI). Maximise the mutual information between a node embedding and a graph summary $g = \text{sigmoid}(\text{mean}_v h_v)$, against a corrupted graph $\tilde{\mathcal{G}}$ produced by row-shuffling the features:

$$\mathcal{L}_{\text{DGI}} = -\frac{1}{2n} \sum_v \left[\log \sigma(h_v^\top W g) + \log(1 - \sigma(\tilde{h}_v^\top W g)) \right]. \quad (17)$$

The corruption is the design decision: shuffling features across nodes keeps the degree distribution and destroys the feature–topology correspondence, so the model must learn what actually goes with what. GraphST and SEDR both use this objective; it is why they need no reconstruction target.

Contrastive learning on augmented views (GRACE/GCA style). Generate two views by masking features and dropping edges, embed both, and pull the same node’s two views together while pushing other nodes apart with an InfoNCE loss:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(z_v^{(1)}, z_v^{(2)})/\tau)}{\sum_u \exp(\text{sim}(z_v^{(1)}, z_u^{(2)})/\tau)}. \quad (18)$$

The biological catch is that the negatives include cells of the same type. In a dataset where 40% of cells are T cells, most “negatives” are false negatives, and the objective spends its capacity separating cells that should be together. Prototype-based variants (scGPCL) and label-aware sampling exist precisely to patch this, and any contrastive single-cell method that does not address it is relying on the temperature to be forgiving.

Masked autoencoding. Mask a fraction of node features (or edges) and reconstruct them, with a scaled-cosine or count-likelihood reconstruction error. This is the graph analogue of the pretraining objective behind the single-cell foundation models in [RL §8], and the family (SpaMask, MAEST) is currently the most active in spatial domain identification.

B.10 Explainability

Three approaches appear in the reading list, in increasing order of trustworthiness.

Attention as explanation. Free, and weak, for the reasons in §B.4.

Gradient saliency. $|\partial \hat{y}_v / \partial x_{v,g}|$, or integrated gradients along a path from a baseline. STAMarker uses saliency over a spatial GNN to nominate domain-specific variable genes. Cheap and faithful to the model, but not to the biology: saliency tells you what the model used, which is only interesting once you trust the model.

Subgraph search (GNNEExplainer, and the family around it). Learn a mask over edges and features that maximises mutual information with the prediction, with a sparsity penalty:

$$\max_M \text{MI}(\hat{y}, (A \odot \sigma(M), X \odot \sigma(F))) - \lambda \|\sigma(M)\|_1. \quad (19)$$

The output is a small subgraph that suffices for the prediction, which is the right *shape* of answer for a driver-gene or disease-module question — GNN-SubNet and CGMega [RL §7.2] both build on this. The caveat is that the explanation is a property of one trained model: re-train with a different seed and the subgraph moves. Report explanations aggregated across seeds, or do not report them as findings.

B.11 Beyond pairwise: hypergraphs, higher-order structure, and time

Hypergraphs. A pairwise edge cannot express a relation among three or more nodes that is not decomposable into pairs. In single-cell genomics the natural example is the 3D genome: a single-cell Hi-C or Pore-C read reports that several loci were in one complex simultaneously, which is a hyper-edge, and reducing it to pairwise contacts loses exactly the multi-way information that motivated the assay. Hyper-SAGNN [RL §2.5] handles this by scoring a candidate hyperedge $e = \{v_1, \dots, v_m\}$ with two embeddings per node: a *static* embedding s_i that ignores the hyperedge and a *dynamic* embedding d_i computed by self-attention over the other members,

$$d_i = \sigma \left(\sum_{j \neq i} \alpha_{ij} W s_j \right), \quad \hat{y}(e) = \sigma \left(\frac{1}{m} \sum_i w^\top (d_i - s_i)^{\odot 2} \right), \quad (20)$$

so that a hyperedge scores highly when membership *changes* each node’s representation — a neat formalisation of “these loci belong together”. The same machinery applies to any multi-way relation: a niche of m cell types, a protein complex, a pathway.

Simplicial and higher-order message passing. More generally, one can pass messages between edges, triangles, and higher simplices rather than only nodes. The single-cell literature has barely touched this, and the honest assessment is that the added expressivity has not yet been shown to pay for its cost on biological data. It is worth knowing the vocabulary exists, because expressivity results in [RL §II.9] are stated in these terms.

Dynamic graphs. Trajectory and velocity models [RL §2.4] need a graph that changes, or a graph whose edges are directed by time. Two constructions appear:

- *Directed velocity graphs.* RNA velocity gives each cell a vector $\dot{x}_c$ in expression space; the transition probability from c to a neighbour u is

$$\pi_{cu} = \frac{\exp(\cos(\dot{x}_c, x_u - x_c)/\tau)}{\sum_{u' \in \mathcal{N}(c)} \exp(\cos(\dot{x}_c, x_{u'} - x_c)/\tau)}, \quad (21)$$

turning the symmetric k NN graph into a Markov chain whose absorbing states are terminal cell fates. DeepVelo learns the velocity field itself with a GNN, so that a cell’s kinetic parameters are estimated using its neighbours — which is the correct response to the fact that per-cell velocity estimates are extremely noisy. - *Temporal snapshots.* Sample-level time points give a sequence of graphs; the standard treatment (from [RL §II.11]) is a GNN for space composed with a recurrence or temporal convolution for time. The reading list deliberately screens out the large traffic-forecasting literature that developed these, but the architectures transfer directly to time-course spatial experiments,

which are becoming common.

Metabolic flux as a graph problem. scFEA is the most unusual member of [RL §2.4]: nodes are metabolites, edges are reaction modules, and the network is constrained by flux balance — the sum of incoming fluxes must match outgoing at each intermediate metabolite. The GNN’s job is to predict module fluxes from expression subject to that constraint, imposed as a penalty:

$$\mathcal{L} = \sum_{m \in \text{intermediates}} \left(\sum_{r \in \text{in}(m)} f_r - \sum_{r \in \text{out}(m)} f_r \right)^2 + \dots \quad (22)$$

This is worth studying as a template for a broader idea: when a mechanistic constraint on the system is known, it can enter as a loss term on a graph rather than as a hard-coded model. Very little of the reading list does this, and it is one of the clearer openings for new work.

Part C — Backbones and objectives

A GNN layer is rarely the whole model. In this literature it is almost always embedded in a generative or self-supervised framework that supplies the loss, and the framework is usually a variational autoencoder with a count likelihood. Understanding that backbone explains more of the published performance differences than the graph layer does.

C.1 Modelling counts properly

UMI counts are non-negative integers, over-dispersed relative to Poisson, and sparse. Three facts determine the likelihood.

Poisson is not enough. For a Poisson variable, mean equals variance. Real scRNA-seq genes have variance far above the mean, because true expression varies across cells of the same type. Model that with a gamma-distributed rate:

$$x \mid \lambda \sim \text{Poisson}(\lambda), \quad \lambda \sim \text{Gamma}(\theta, \theta/\mu) \implies x \sim \text{NB}(\mu, \theta), \quad (23)$$

$$p(x \mid \mu, \theta) = \frac{\Gamma(x + \theta)}{\Gamma(\theta) x!} \left(\frac{\theta}{\theta + \mu} \right)^\theta \left(\frac{\mu}{\theta + \mu} \right)^x, \quad \text{Var}(x) = \mu + \frac{\mu^2}{\theta}.$$

θ is the inverse dispersion: $\theta \rightarrow \infty$ recovers Poisson, small θ means heavy over-dispersion. It is standard, and correct, to fit one θ_g per gene rather than per cell.

Zero-inflation is a modelling choice, not a fact. The ZINB likelihood adds a point mass at zero,

$$p_{\text{ZINB}}(x) = \pi \delta_0(x) + (1 - \pi) p_{\text{NB}}(x \mid \mu, \theta), \quad (24)$$

and much of the single-cell GNN literature ([RL §2.1]: scGNN, scDSC, the ZINB-graph-autoencoder family) uses it by default. The evidence that UMI-based protocols need it is weak — negative binomial alone fits droplet data well, and the extra π_{cg} head mostly absorbs variance the NB would have handled. Keep ZINB for read-based or plate-based protocols and for scATAC binarised counts; for 10x UMI data, fit NB first and check whether π does anything.

Library size must not be learned away. Sequencing depth varies several-fold across cells for purely technical reasons. The standard decomposition separates it from biology:

$$\mu_{cg} = \ell_c \rho_{cg}, \quad \sum_g \rho_{cg} = 1, \quad (25)$$

with ρ_c produced by a softmax output layer and ℓ_c either the observed total count or a per-batch latent with a log-normal prior. This is the mechanism by which scVI-style models avoid needing a normalisation step at all — and it is exactly what is lost when a method trains a graph autoencoder

with mean-squared error on log-normalised data, as a good fraction of [RL §2.1] does. MSE on $\log(1 + x)$ assumes homoscedastic Gaussian noise on a transformed count, which is wrong in a way that systematically underweights highly expressed genes.

Other modalities. scATAC peaks: Bernoulli on binarised data, or NB on fragment counts. CITE-seq protein (ADT): NB with a background component, since ambient antibody gives a non-zero floor. Spatial spot data (Visium): NB, with the caveat that a spot is 1–10 cells, so the “expression” being modelled is a mixture — which is what the deconvolution methods of [RL §4.2] exist to undo.

C.2 The variational autoencoder backbone

a Count VAE (scVI-style): the backbone

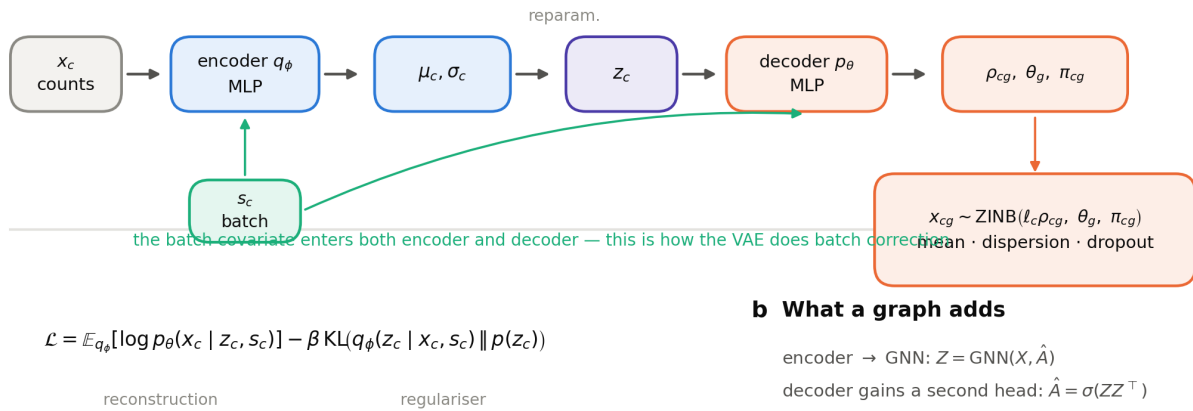


Figure 7: The count VAE, and the two places a graph enters it.

The ELBO. With latent z_c and prior $p(z) = \mathcal{N}(0, I)$, the marginal likelihood is intractable, so maximise a lower bound. For any q_ϕ ,

$$\log p_\theta(x_c) = \log \int p_\theta(x_c | z) p(z) dz \geq \mathbb{E}_{q_\phi(z|x_c)}[\log p_\theta(x_c | z)] - \text{KL}(q_\phi(z | x_c) \| p(z)), \quad (26)$$

the gap being $\text{KL}(q_\phi(z | x_c) \| p_\theta(z | x_c)) \geq 0$. The first term is reconstruction, evaluated with (23) or (24); the second is the regulariser that makes the latent space continuous and interpolable — the property that lets you cluster in it and lets a decoder generalise to a batch it has not seen.

Reparameterisation. Sampling blocks gradients, so write $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$; now the randomness is in ϵ and ∇_ϕ passes through. With diagonal Gaussians the KL is closed-form:

$$\text{KL} = \frac{1}{2} \sum_{j=1}^d (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1). \quad (27)$$

Conditioning on batch. Passing the batch label s_c (one-hot) to both encoder and decoder makes the latent conditionally independent of batch: $q_\phi(z \mid x, s)$, $p_\theta(x \mid z, s)$. The decoder can use s to explain batch-driven variation, so z need not encode it. This is the whole of scVI’s batch correction, it costs nothing, and it is strictly better behaved than post-hoc correction of an embedding.

Posterior collapse. If the decoder is powerful enough to reconstruct x without z , the optimiser drives $q_\phi(z \mid x) \rightarrow p(z)$ and the latent goes silent. Symptoms: KL near zero, embeddings that look like noise, clusters that do not separate. Fixes: KL warm-up (anneal β from 0 to 1 over the first few epochs), free bits (floor the per-dimension KL), or a weaker decoder. In graph-augmented models this failure is easy to misread as “the graph did not help”.

β and the trade-off. Scaling the KL by β trades reconstruction against latent regularity. $\beta > 1$ gives smoother, more disentangled, less faithful latents; $\beta < 1$ the reverse. For integration tasks a mildly larger β usually improves batch mixing at some cost in biological conservation — which is precisely the axis the scIB benchmark measures (§E.1).

C.3 Graph autoencoders

Where the VAE reconstructs features, the graph autoencoder reconstructs structure. Encode with a GNN and decode with an inner product:

$$Z = \text{GNN}(X, \hat{A}) \in \mathbb{R}^{n \times d}, \quad \hat{A}_{ij} = \sigma(z_i^\top z_j), \quad (28)$$

$$\mathcal{L}_{\text{GAE}} = - \sum_{(i,j) \in \mathcal{E}} \log \sigma(z_i^\top z_j) - \sum_{(i,j) \in \mathcal{E}^-} \log(1 - \sigma(z_i^\top z_j)). \quad (29)$$

The variational form (VGAE) puts $q(z_i \mid X, A) = \mathcal{N}(\mu_i, \text{diag } \sigma_i^2)$ with GNN-produced parameters and adds the KL of (27).

Three practical points. **Negative sampling** is required: $\mathcal{E}$ has $O(nk)$ entries and the non-edges $O(n^2)$, so sample $\mathcal{E}^-$ at a fixed ratio (1:1 to 1:5) per epoch, and be aware that the ratio changes the calibration of $\hat{A}$, though not usually the ranking. **The inner-product decoder is symmetric and transitive-ish** — it can only express similarity structure that embeds in $\mathbb{R}^d$ — so it cannot represent a directed regulatory edge; use the bilinear form $\sigma(z_i^\top W z_j)$ with asymmetric W when direction matters (§D.6). And **reconstructing A is a strange objective for a cell graph**, since A was itself constructed from X moments earlier: the model is being asked to recover a k NN structure it could compute directly. That circularity is why the better single-cell GAEs (scGAE, CellVGAE, and the ZINB-GAE family) reconstruct *both* A and X , with the count likelihood carrying the real information and the graph term acting as a topology-preserving regulariser.

C.4 Aligning distributions

Integration means making the embeddings of different batches, modalities, or datasets occupy the same region of latent space while preserving biological differences. Four mechanisms recur.

Conditional decoding, as above: cheapest, works when the batch effect is roughly a shift the decoder can express.

Adversarial alignment. A discriminator D_ω tries to predict the batch (or modality) from z ; the encoder is trained to defeat it:

$$\min_{\phi} \max_{\omega} \mathbb{E}_{z \sim q_{\phi}} [\log D_{\omega}(z) + \log (1 - D_{\omega}(z'))], \quad (30)$$

implemented with a gradient-reversal layer so both sides train in one pass. This is what GLUE uses to bring RNA and ATAC posteriors together (§D.2). It aligns distributions globally, which is its strength and its danger: nothing in (30) requires that a T cell land on a T cell, only that the two clouds overlap, so an adversarial integrator can align cell types that should not be aligned when composition differs strongly between datasets.

Maximum mean discrepancy. A kernel two-sample statistic added directly to the loss,

$$\text{MMD}^2 = \left\| \frac{1}{n_1} \sum_i \varphi(z_i) - \frac{1}{n_2} \sum_j \varphi(z'_j) \right\|_{\mathcal{H}}^2, \quad (31)$$

with a Gaussian kernel. No inner optimisation, no instability, weaker at correcting nonlinear batch effects; a reasonable first choice when adversarial training will not converge.

Optimal transport and matching. Compute a soft correspondence between two sets of cells by minimising transport cost under marginal constraints, then use it as a supervision signal. This is the formal version of the anchor idea that Seurat introduced, and it appears in the alignment methods of [RL §4.4] where two tissue slices must be put in register.

C.5 Clustering heads

Many methods in [RL §2.1] and [RL §4.1] are unsupervised and end in a clustering step. Two designs:

Post-hoc. Take Z , build a k NN graph on it, run Leiden. Simple, reproducible, and the resolution parameter is then the thing that decides the number of clusters — which means the comparison against a baseline is only fair if the baseline’s resolution was tuned the same way. A depressing amount of the reported ARI variation in this literature is resolution tuning.

Joint (DEC-style). Learn cluster centroids $\{\nu_k\}$ jointly with the embedding. Assign softly with a Student- t kernel, then sharpen:

$$q_{ck} = \frac{(1 + \|z_c - \nu_k\|^2/\alpha)^{-\frac{\alpha+1}{2}}}{\sum_{k'} (1 + \|z_c - \nu_{k'}\|^2/\alpha)^{-\frac{\alpha+1}{2}}}, \quad p_{ck} = \frac{q_{ck}^2 / \sum_c q_{ck}}{\sum_{k'} q_{ck'}^2 / \sum_c q_{ck'}}, \quad (32)$$

$$\mathcal{L}_{\text{clu}} = \text{KL}(P\|Q).$$

The target P sharpens Q and normalises by cluster frequency, so large clusters do not swallow small ones. SEDR, scDSC, ADEPT and others use exactly this. It is self-reinforcing — the model is trained towards its own confident predictions — so initialisation matters enormously, and k must be fixed in advance, which for spatial domain identification means the number of domains is an input, not an output. Papers that tune k per dataset against the ground truth and then report ARI have reported an upper bound, not a result.

Part D — The applied architectures

D.1 The cell-graph template

Nineteen of the papers in [RL §2.1] share one architecture. Written once:

$$A = k\text{NN}(\text{PCA}(X)); \quad Z = \text{GNN}(X, \hat{A}); \quad \mathcal{L} = \underbrace{\mathcal{L}_{\text{recon}}(X, \hat{X})}_{\text{ZINB or MSE}} + \lambda_1 \underbrace{\mathcal{L}_{\text{graph}}(A, \hat{A})}_{\text{GAE term}} + \lambda_2 \underbrace{\mathcal{L}_{\text{clu}}(P, Q)}_{\text{DEC term}}. \quad (33)$$

The published variation is which terms are present, which GNN fills the middle, and how the graph was built:

Method	Encoder	Reconstruction	Extra term	Distinguishing idea
scGNN	GNN + AE ensemble	LTMG-regularised	iterative	re-estimates the graph each iteration
scGAE	GAT	MSE	topology loss	preserves manifold topology explicitly
CellVGAE	GAT	VGAE	—	variational, attention as interpretation
scDSC	GCN + AE	ZINB	DEC	fuses AE and GCN representations layer-wise
graph-sc	GCN on cell–gene graph	ZINB	—	bipartite rather than cell–cell
scGPCL	GCN	—	prototype contrastive	fixes the false-negative problem in §B.9
scGCC	GCN	—	contrastive + augmentation	neighbourhood augmentations

Two structural observations are worth more than the table. First, when the graph is built from PCA of X and the model reconstructs X , the graph adds no information that was not in X — it adds an *inductive bias*, namely that neighbouring cells should have similar representations. That bias is real and useful (Figure 5, right panel, shows it is worth a factor of three in cluster separability at

two hops), but it is a smoother, not a new measurement, and it is why the honest ablation is the permuted graph of §A.4 rather than “no graph”.

Second, iterative methods that re-estimate A from Z and then re-train — scGNN, ADEPT, the consensus autoencoders — are performing a form of self-training on the graph. They usually improve ARI and always risk confirming their own errors: a cell assigned early to the wrong neighbourhood gets edges that keep it there. Where these methods report improvements, look for whether the improvement survives a fixed number of iterations chosen without reference to the ground truth.

Annotation and label transfer [RL §2.2] change (33) in one place: the loss becomes cross-entropy on labelled nodes, and the interesting design question moves to how reference and query cells share a graph. scGCN builds a joint graph across datasets via mutual nearest neighbours and then propagates labels through it; CAME builds a heterogeneous graph with cells and genes as separate node types so that homologous genes bridge species. Both are, at heart, semi-supervised node classification, where the graph’s job is to carry labels across the batch boundary — which makes edge purity across that boundary (§A.4) the diagnostic that predicts success.

D.1.1 Imputation and denoising

The imputation methods of [RL §2.3] take (33) and change what the reconstruction targets. The premise is that a cell’s neighbours are technical replicates of its true expression state, so pooling over the graph recovers the signal that dropout removed:

$$\hat{x}_c = \sum_{u \in \mathcal{N}(c) \cup \{c\}} \hat{A}_{cu} x_u \quad (\text{the trivial version}), \quad \hat{x}_c = \text{decoder}\left(\text{GNN}(X, \hat{A})_c\right) \quad (\text{the published version}).$$

GraphSCI couples a GCN over a gene–gene graph with an autoencoder over cells, so information flows both across similar cells and across co-expressed genes. scGGAN adds an adversarial discriminator on the imputed matrix, which sharpens the output distribution but makes the result harder to interpret as an expectation.

Two warnings that the benchmarking literature has established firmly. First, **imputation is smoothing**, and smoothing inflates gene–gene correlations: after imputation, a co-expression network computed on the imputed matrix will contain edges that are artefacts of the graph used to impute. Never impute before network inference. Second, **evaluation by masking is circular** when the mask is applied to observed non-zeros and the metric is reconstruction of those entries — the model is being scored on recovering exactly the smoothness it assumed. The better evaluations use a downstream task (clustering, DE recovery against a bulk reference) or paired protocols where a deeper measurement of the same cells exists.

D.1.2 What the graph does for trajectories

Trajectory inference had graph methods long before deep learning: most classical tools build a k NN graph and compute a minimum spanning tree, diffusion pseudotime, or a partition-based graph abstraction over it. The GNN contribution [RL §2.4] is to make the *representation* on which that graph is built learnable, and to let neighbouring cells share statistical strength when estimating a

per-cell quantity — velocity, flux, differentiation potential — that is far too noisy to estimate cell by cell.

That framing sets the correct expectation. A GNN does not remove the fundamental ambiguity of trajectory inference: pseudotime is identified only up to the assumption that transcriptional similarity tracks temporal proximity, and no amount of message passing tests that assumption. What it can do is reduce variance, which is worth having, and it is why the honest evaluations in this subsection report the stability of the inferred trajectory across seeds and subsamples rather than agreement with a single annotated lineage.

D.2 Guidance graphs: integration without shared features

The hardest integration problem is unpaired multi-omics: RNA measured on one set of cells, ATAC on another, no shared cells and no shared features. The guidance-graph idea solves it by refusing to align cells at all, and aligning *features* instead — through a graph that encodes what is known about how features relate.

a Guidance-graph integration (GLUE): unpaired modalities linked through a feature graph

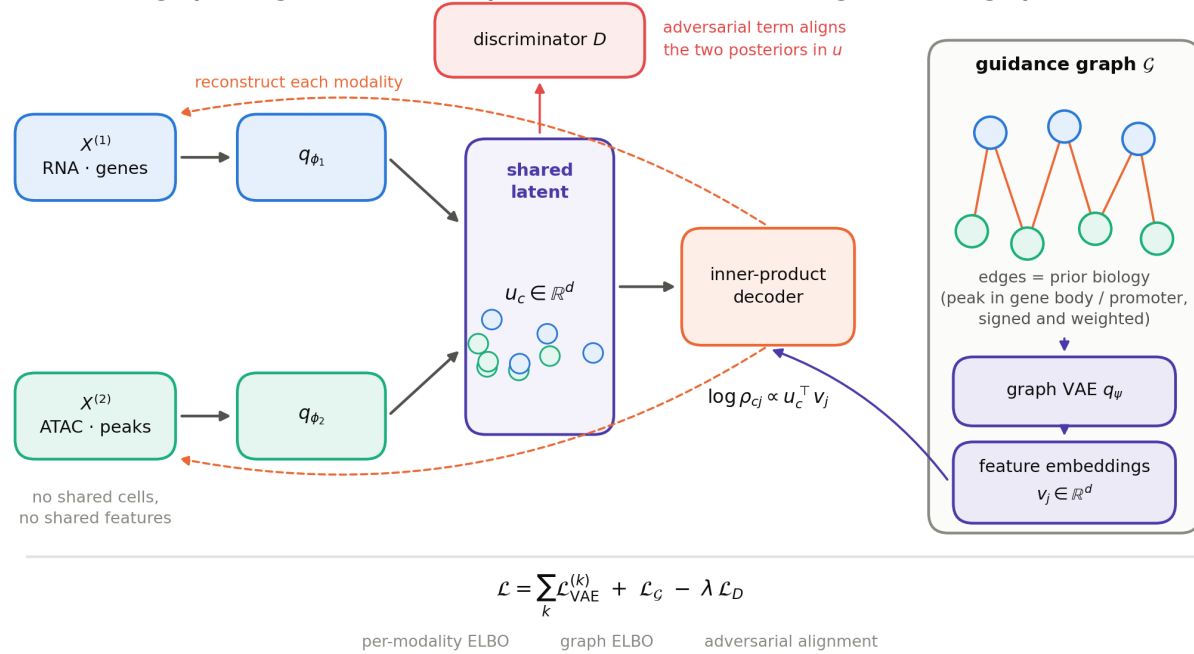


Figure 8: GLUE: two modality-specific VAEs share a latent space; a graph VAE over the feature graph supplies the decoder weights; a discriminator aligns the posteriors.

The construction. Let k index modalities, $x_c^{(k)}$ the measurement of cell c , and $\mathcal{G}$ a graph whose nodes are all features of all modalities, with edges encoding prior regulatory relationships (a peak inside a gene body, signed positive; a peak in a known silencer, signed negative).

1. **Cell encoders.** $q_{\phi_k}(u_c | x_c^{(k)}) = \mathcal{N}(\mu_k(x_c), \sigma_k(x_c))$, one per modality, all mapping into the same d -dimensional latent space.

2. **Feature encoder.** A graph VAE on $\mathcal{G}$ gives each feature j an embedding $v_j \in \mathbb{R}^d$ in that same space, with the graph reconstructed by $p(e_{jj'}) \propto \sigma(v_j^\top v_{j'})$ weighted by the prior sign.
3. **Decoder.** The likelihood of a measurement is generated from the inner product of the cell and feature embeddings:

$$\log \rho_{cj} \propto u_c^\top v_j + b_j, \quad x_{cj} \sim \text{NB}(\ell_c \text{softmax}_j(\rho_{cj}), \theta_j). \quad (34)$$

4. **Alignment.** A discriminator D predicts the modality from u_c ; the encoders are trained adversarially against it (30).

$$\mathcal{L} = \sum_k \mathcal{L}_{\text{VAE}}^{(k)} + \mathcal{L}_{\mathcal{G}} - \lambda \mathcal{L}_D. \quad (35)$$

Why this works when anchor methods fail. Anchor-based integration needs a shared feature space — gene activity scores, computed by summing ATAC fragments over gene bodies — and gene activity is a crude proxy that fails for distal enhancers and for genes regulated by chromatin state rather than accessibility. GLUE never forms it. The link between modalities is carried entirely by the geometry of the feature graph: if peak p is connected to gene g , their embeddings are pulled together, so cells with high p and cells with high g end up in the same latent region. The prior does the work that the shared feature space used to do, and the prior is inspectable — you can see which edges the model relied on, and GLUE’s regulatory-inference output is exactly the posterior over those edges.

Where it breaks. The guidance graph is a hypothesis. Wrong edges produce confidently wrong alignments, and the adversarial term will happily overlay two populations that share no biology if that is the only way to fool D . The published safeguard — check that the integrated space still separates known cell types, and that the model’s re-weighted edges agree with held-out Hi-C or eQTL evidence — is a check on the prior, not on the fit.

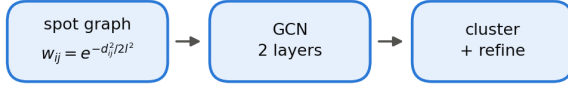
Relatives. SIMBA drops the VAE and embeds cells, genes, peaks, and motifs as nodes of one heterogeneous graph with a knowledge-graph objective (TransE-style scoring of triples), giving a single space in which a nearest-neighbour query answers “which features mark this cell”. scMoGNN treats the cell–feature bipartite graph directly and won the NeurIPS multimodal benchmark by, essentially, careful feature engineering on the edges. DeepMAPS runs a heterogeneous graph transformer on the cell–gene graph and reads gene modules out of the attention. UnitedNet frames the same data as multi-task learning and gets interpretability from task-specific heads. BABEL — no graph at all, just paired encoders/decoders trained on matched cells — is the control that shows how much of this machinery a paired dataset actually needs.

D.3 Spatial GNNs

Spatial transcriptomics is where graph methods have their cleanest justification: the graph comes from physical coordinates, not from the expression the model is fitting, so message passing adds genuinely external information. [RL §4.1] contains 22 methods for one task — identifying spatial domains — and the differences between them are almost entirely in the four dimensions below.

a SpaGCN

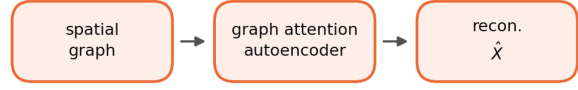
expression + histology fixed into one adjacency



$$d_{ij}^2 = (x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2$$

histology enters as a third coordinate z ; the graph is not learned**b STAGATE**

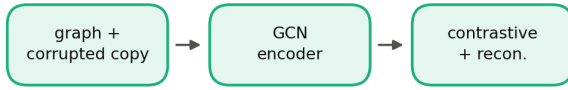
attention learns which neighbours to trust

 α_{ij} learned; optional pruning by pre-cluster label

attention is the fix for smoothing across a domain boundary

c GraphST

self-supervision instead of a reconstruction target

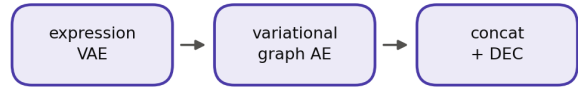


$$\mathcal{L}_{SSL} = -\log \sigma(h_i^\top g) - \log(1 - \sigma(\tilde{h}_i^\top g))$$

positive = real neighbourhood, negative = shuffled features

d SEDR

two encoders, one embedding



$$z = [z_{\text{expr}} \parallel z_{\text{graph}}]$$

keeps a non-graph channel, so a bad graph cannot erase the signal

Figure 9: Four representative spatial architectures. The differences are in the graph, the objective, and where histology enters — not in the message-passing layer.

SpaGCN: put everything in the graph. Build a weighted graph on a three-dimensional coordinate (x, y, z) , where z is derived from the histology image’s local RGB statistics and rescaled so that it contributes comparably to the physical axes:

$$w_{ij} = \exp\left(-\frac{d_{ij}^2}{2l^2}\right), \quad d_{ij}^2 = (x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2, \quad (36)$$

then run a plain 1–2 layer GCN and cluster with a DEC head, followed by a spatial refinement step that reassigns each spot to the majority label of its neighbours. Nothing about the graph is learned. The bandwidth l is set so that each spot’s total weight to its neighbours hits a target — the one hyperparameter that matters. SpaGCN’s strength is that it is fast, stable, and hard to break; its weakness is that a fixed kernel smooths across a domain boundary exactly as readily as within a domain.

STAGATE: learn the edge weights. Keep the graph binary and geometric, and let a graph attention autoencoder decide how much each neighbour contributes:

$$h_i^{(\ell)} = \sigma\left(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \alpha_{ij} W^{(\ell)} h_j^{(\ell-1)}\right), \quad \hat{x}_i = \text{decoder}(h_i),$$

trained to reconstruct expression, with tied encoder/decoder weights. The optional cell-type-aware step pre-clusters, then prunes edges between spots assigned to different pre-clusters, which is a hard version of what attention is supposed to do softly. Attention is the correct answer to the boundary

problem in principle; in practice it needs enough signal to learn from, and on low-depth Visium sections the learned α often ends up near-uniform, at which point STAGATE is SpaGCN with more parameters.

GraphST: replace reconstruction with self-supervision. Augment the graph by shuffling features across spots to make a corrupted view, encode both with a GCN, and train the DGI objective of (17) alongside a reconstruction term; then refine with a smoothing step over spatial neighbours. The self-supervised objective is what allows GraphST to also do vertical (slice-to-slice) integration and deconvolution with the same embedding, which is its real selling point — one representation, three tasks.

SEDR: keep a non-graph channel. Run a deep autoencoder on expression and a variational graph autoencoder on the spatial graph, concatenate the two latents, and cluster the concatenation. The architecture is redundant by design: if the spatial graph is uninformative — a tissue with no spatial organisation, or coordinates with registration error — the expression channel still carries the signal. That redundancy is why SEDR is a good default on unfamiliar data, and why its ceiling on well-structured tissue is lower than the attention-based methods’.

Multi-view methods (Spatial-MGCN, SpaNCMG, and the “multi-view GCN” family) formalise the same intuition: build both a spatial graph and a feature graph, encode each, and fuse with attention or a shared-plus-private decomposition. The consistent empirical finding across [RL §4.1] is that two views beat one, and that the gain comes mostly from the model’s ability to *ignore* the spatial graph where it is misleading.

D.3.1 The other spatial tasks

Deconvolution [RL §4.2]. A Visium spot contains several cells, so the task is to estimate proportions $\pi_s \in \Delta^{K-1}$ over reference cell types. STdGCN builds two graphs — a spot–spot spatial graph and a spot–pseudospot graph linking real spots to simulated mixtures from a reference — and propagates over both, so that a spot’s proportions are informed by its neighbours’ as well as by the reference. This is a genuine use of the graph: proportions are spatially autocorrelated, and enforcing that smooths away the sampling noise that dominates low-count spots. The evaluation trap is that simulated pseudospots are generated from the same reference used to score, and cross-protocol validation is rare.

Histology to expression [RL §4.3]. Predict expression from an H&E patch. Hist2ST composes a convolutional patch encoder, a transformer over patches, and a GNN over the spot neighbourhood graph, with a ZINB output head. The graph’s role is to impose spatial consistency on the predictions. The reported correlations look impressive until you notice how much of the achievable correlation comes from predicting the tissue-region mean: the right baseline is a model that predicts each spot’s expression from its neighbours’ *observed* expression, and papers that omit it are measuring spatial autocorrelation, not image-to-expression prediction.

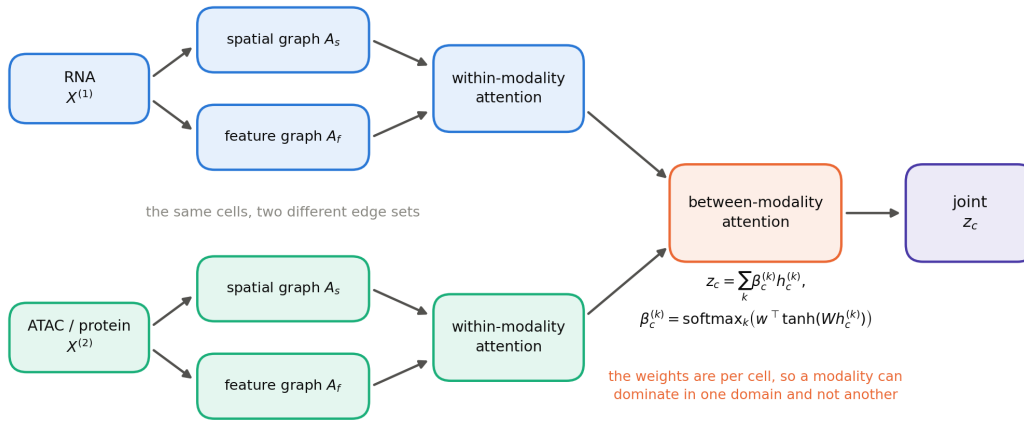
Cross-slice alignment [RL §4.4]. STAligner learns a shared embedding across slices with a triplet loss over mutual nearest neighbours identified in embedding space, iterating between alignment and re-embedding; SANTO frames the same problem as a rigid or non-rigid registration solved by alternating correspondence and transformation estimation. Both are the spatial analogue of anchor-based integration, with geometry constraining the correspondence.

Segmentation [RL §4.7]. Segger treats imaging-based data (Xenium, MERFISH) as a heterogeneous graph of transcripts and nuclei, and assigns each transcript to a cell by link prediction. This reframing is the most consequential idea in the subsection, because it makes segmentation differentiable and lets it use molecular identity rather than image morphology alone. It is also upstream of everything else: every method in [RL §4] that starts from a cell-by-gene matrix inherits whatever errors segmentation made, and those errors are not small.

D.4 Spatial multi-omics

When two modalities are measured on the same tissue section, each cell has two feature vectors and at least two natural graphs, and the modelling question becomes how to weight them per cell rather than globally.

a Spatial multi-omics: two graphs per modality, two levels of attention



b Mosaic case batch 1 has RNA+ATAC, batch 2 has RNA+protein, batch 3 has RNA only — the shared modality is the bridge that carries the others into one space

Figure 10: Two levels of attention: within a modality across its two graphs, then between modalities per cell.

SpatialGlue. For each modality k , build a spatial neighbour graph A_s and a feature-similarity graph A_f ; encode with a GNN over each; combine them with a within-modality attention; then combine modalities with a between-modality attention:

$$z_c = \sum_k \beta_c^{(k)} h_c^{(k)}, \quad \beta_c^{(k)} = \text{softmax}_k(w^\top \tanh(W h_c^{(k)})). \quad (37)$$

Because β is indexed by c , a cell in a region where chromatin is uninformative can down-weight ATAC while its neighbour does the opposite. The attention weights are also the method’s interpretability story — a map of which modality dominates where — and, per §B.4, that map deserves the same scepticism as any other attention-as-explanation claim, though here it is at least checkable against known tissue biology.

MultiGATE couples the two modalities through a shared attention structure and reads regulatory relationships out of the cross-modality weights. **COSMOS** builds a single graph over the joint measurement and optimises a cooperative objective. **SpaMosaic** handles the mosaic case — batches

that measure different, partially overlapping modality subsets — by using the modality present in every batch as a bridge, an idea imported from mosaic single-cell integration and, at the time the reading list was compiled, the most practically useful architecture in this subsection because real spatial multi-omic experiments almost never share a common panel.

D.5 Cell-cell communication

The natural question — does the presence of cell type B near cell A change what A expresses? — has an unusually clean graph formulation, and one method states it as a conditional model rather than a scoring heuristic.

NCEM. Regress each cell’s expression on its own type and the composition of its neighbourhood:

$$\mathbb{E}[x_c] = f(t_c, \{t_u : u \in \mathcal{N}(c)\}), \quad (38)$$

with f either a linear model on the neighbourhood type-composition vector or a GNN. The quantity of interest is the *gain* in fit from the neighbourhood term over the cell-type-only model, per gene and per type pair. Because the null model is explicit, the result is a statistically meaningful statement about how much of a cell’s variance its surroundings explain — which is what makes NCEM the reference point for the subsection, whatever its performance on any leaderboard.

GCNG takes the complementary approach: put genes on the nodes and use the spatial graph to learn which ligand–receptor pairs co-vary across neighbouring cells, converting a communication question into supervised edge classification against a curated LR database. **DeepLinc** reconstructs the interaction graph itself with a variational graph autoencoder, so that edges absent from the observed neighbourhood graph can be predicted *de novo* — attractive, and dependent on the assumption that latent similarity implies interaction, which is exactly what one wanted to test.

The field-wide caveat: proximity is not communication, and every method here is estimating an association between neighbourhood composition and expression. Ligand–receptor annotation adds a mechanistic prior but not a causal identification. Perturbation data is what would settle it, and almost none of the papers in [RL §5] have any.

D.6 Gene regulatory networks

GRN inference on a graph is link prediction: nodes are genes, node features are expression profiles across cells, and the model scores candidate regulatory edges.

The architecture. GENELink is representative. Node features are the gene’s expression vector across all n cells (so the “feature dimension” is the number of cells, usually reduced first); the candidate graph comes from a curated TF–target database or from motif scanning; a GAT encoder produces z_i ; and a bilinear decoder scores the ordered pair:

$$s_{ij} = \sigma(z_i^\top W z_j), \quad W \in \mathbb{R}^{d \times d} \text{ asymmetric.} \quad (39)$$

a GRN inference as link prediction on a gene graph

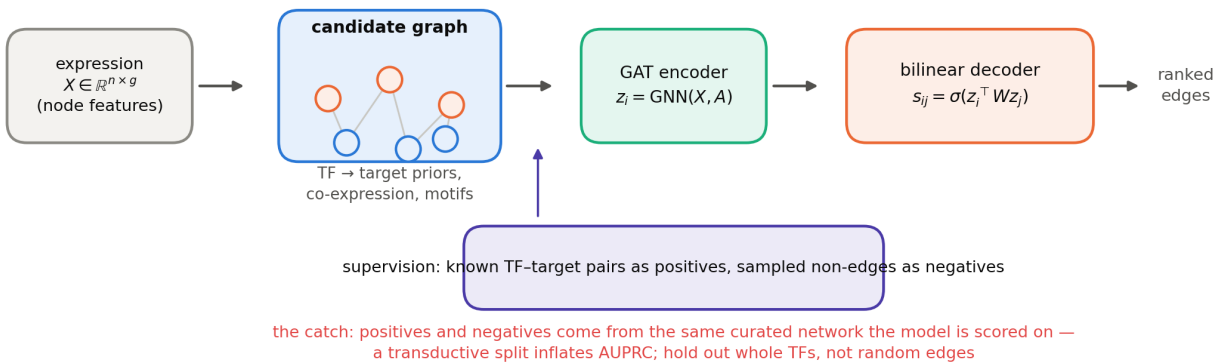


Figure 11: GRN inference as supervised link prediction, and the evaluation trap that comes with it.

Asymmetry is the whole point — regulation is directed, and the symmetric inner product of (28) cannot express it. Training uses known TF–target pairs as positives and sampled non-edges as negatives, with a cross-entropy or margin loss. GNNLink swaps the encoder for a different message-passing scheme; scMGATGRN adds multiple views; GMFGRN combines matrix factorisation with the GNN; CEFCON goes further and asks which regulators drive a fate transition, by combining a network-propagation step with a control-theoretic notion of driver nodes.

The evaluation trap, stated plainly. The positives come from a curated network; the negatives are sampled from the same universe; and the test set is a random split of those same edges. A model that learns “genes adjacent to hub TFs in the curated network are likely to be adjacent to hub TFs” will score well without learning anything about regulation. Two symptoms are diagnostic: performance that barely degrades when expression features are replaced with random vectors, and AUPRC that collapses when the split is made by whole TFs (all edges of a held-out TF removed) rather than by random edges. The BEELINE benchmark and its successors make this point repeatedly; the honest protocols in this subsection hold out TFs, evaluate on an independent network (ChIP-seq derived, when the training network was motif-derived), and report early precision rather than global AUROC, which is dominated by the vast majority of true negatives.

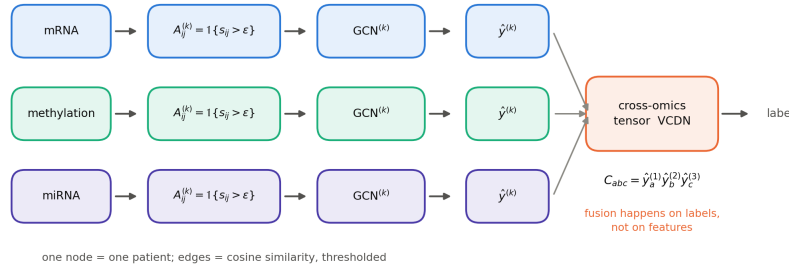
What the graph is actually for. In most of these methods the candidate graph is both the prior and the search space, which conflates two roles. A cleaner design keeps them apart: use the prior graph as the message-passing substrate and score *all* pairs, so that predicted edges outside the prior are possible. Methods that only score within the prior can, by construction, never discover a new regulator.

Chromatin as evidence. The scATAC-based methods in this subsection are the most promising direction, because peak accessibility plus motif presence gives an edge prior that is not derived from the expression correlation being modelled. That independence is what breaks the circularity above.

D.7 Patient-level multi-omics

The largest section of the reading list (48 papers) is also the one where the graph is doing the least work, and it is worth being explicit about why.

a Patient-similarity networks (MOGONET pattern)



b Driver-gene discovery (EMOGI pattern):

nodes are genes on a PPI network, node features are the per-gene multi-omic vector (mutation rate, CNV, methylation, expression) — node classification, not graph-level

Figure 12: Per-omic patient graphs, per-omic GCNs, and fusion at the label level.

MOGONET and the VCDN pattern. Build one patient-similarity network per omic (3); train one GCN per omic to produce class probabilities $\hat{y}^{(k)} \in \Delta^{C-1}$; then fuse with a View Correlation Discovery Network, which forms the outer product of the per-omic predictions and learns over it:

$$C_{abc} = \hat{y}_a^{(1)} \hat{y}_b^{(2)} \hat{y}_c^{(3)}, \quad \hat{y} = \text{MLP}(\text{vec}(C)). \quad (40)$$

Fusion happens on the *labels*, not the features, which is what keeps the parameter count survivable with a few hundred patients. Variants swap in attention (MOGAT, omicsGAT), add prior-knowledge graphs (SUPREME, MODILM), or make the graph learnable (adaptive graph learning). The architectural space has been thoroughly explored; the statistical problem has not.

The statistical problem. With $n \approx 300$ patients, a two-layer GCN, and $p \approx 20,000$ features per omic, the model has orders of magnitude more parameters than samples. The graph is built from the same features used as node inputs, so message passing mixes information across patients whose similarity was computed from those features — a transductive setup in which the test patient’s own features helped define its neighbourhood. Under repeated stratified cross-validation with all preprocessing inside the fold, several independent re-evaluations find these methods approximately tied with regularised logistic regression on concatenated omics, and sometimes behind it. That result does not make the section worthless, but it does mean any new method here needs: preprocessing inside the fold, a linear baseline on the same folds, repeated splits with variance reported, and a permuted-graph control.

Driver-gene discovery [RL §7.2] is the better-posed cousin. Here nodes are genes on a PPI network — an edge set that is genuinely external — node features are per-gene multi-omic summaries (mutation frequency, copy number, methylation, expression across a cohort), and the task is node classification against known cancer genes. EMOGI established the pattern; MODIG adds multiple gene-similarity views; CGMega and GNN-SubNet add explainability so that the output is a module rather than a ranked list. The graph here is doing real work: a gene’s neighbours in the interactome

are informative about its role in a way that is not recoverable from its own features. The residual difficulty is label quality — “known cancer genes” is an evolving, biased, incomplete set, so a model rewarded for recovering it is partly rewarded for recovering ascertainment bias.

Survival and drug response [RL §7.3–7.4] add a Cox partial-likelihood head,

$$\mathcal{L}_{\text{Cox}} = - \sum_{i: \delta_i=1} \left(\hat{r}_i - \log \sum_{j \in R(t_i)} e^{\hat{r}_j} \right), \quad (41)$$

with $\hat{r}$ the predicted log-risk and $R(t_i)$ the risk set. Everything above about sample size applies, with the extra difficulty that concordance indices in the 0.6–0.7 range are typical and differences of 0.01 between methods are not distinguishable at these cohort sizes without a bootstrap that most papers do not report.

D.8 Where foundation models fit

The single-cell foundation models of [RL §8] — scGPT, Geneformer, scFoundation, scBERT — are transformers pretrained on tens of millions of cells, and they are usually presented as an alternative to the graph methods in this companion. The more useful framing is that they make a different bet about where the inductive bias should come from.

A GNN’s bias is **explicit and local**: you assert the graph, and the model smooths over it. A transformer’s bias is **learned and global**: attention over genes (Geneformer ranks genes and attends over the ranking; scGPT attends over gene tokens with expression-value embeddings) discovers relationships from data volume rather than being told them. The trade is the familiar one — the graph is cheap, interpretable, and wrong wherever the prior is wrong; the transformer needs enormous pretraining data and gives back a representation whose behaviour on a small new dataset is hard to predict.

Three concrete points of contact. First, attention over genes *is* a dense learned graph: a fine-tuned scGPT’s attention maps have been used, with the usual caveats of §B.4, as a GRN estimate. Second, the critical literature — the re-evaluations finding that foundation-model embeddings underperform simpler baselines on several downstream tasks under matched preprocessing — applies the same discipline this companion asks for everywhere: match the preprocessing, include the trivial baseline, report variance. Third, the two approaches compose: use a foundation-model embedding as node features X and keep the graph for local structure. Whether that composition beats either alone is, at the time of writing, unsettled and worth an experiment.

Part E — Evaluating any of this

E.1 Metrics, and what they actually measure

Clustering. Adjusted Rand Index compares two partitions, corrected for chance:

$$\text{ARI} = \frac{\sum_{ij} \binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}{\frac{1}{2} \left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2} \right] - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}. \quad (42)$$

ARI is dominated by large clusters; NMI is more forgiving of splitting a cluster; neither says anything about whether the rare population you care about was found. Report both, and report per-cell-type F1 for the types that matter.

Integration. The scIB suite separates two axes that are in tension: *batch correction* (kBET, graph iLISI, PCR comparison, silhouette by batch) and *biological conservation* (NMI/ARI against cell type, isolated-label F1, cell-cycle conservation, trajectory conservation). Any method can win on one axis by sacrificing the other — an encoder that maps everything to a point has perfect batch mixing. The composite score is a weighted mean (commonly 0.6 bio / 0.4 batch), and where a paper reports only one axis, or reports a composite with its own weights, the comparison is uninterpretable.

Spatial domains. ARI against pathologist annotation is standard and inherits that annotation’s coarseness. Complement it with spatial coherence measures — Moran’s I on the cluster indicator, or the fraction of a spot’s neighbours sharing its label — which detect the common failure of a method producing the right *proportions* in a spatially incoherent scatter. Note that any method with a spatial smoothing step will score well here by construction; the metric rewards the mechanism, so it cannot also validate it.

Link prediction (GRN, communication). AUROC is nearly useless under the extreme class imbalance of a gene network ($\sim 10^4$ true edges among $\sim 10^8$ pairs); AUPRC is better; early precision (precision in the top- k ranked edges, k set to the number of true positives) is what corresponds to how the output is used.

Survival. Harrell’s concordance index, with a bootstrap confidence interval. Differences below 0.02 at $n < 500$ are noise.

E.2 The ablations that decide whether a paper is about graphs

Four experiments, in increasing order of how much they hurt:

1. **No graph.** Replace the GNN with an MLP of matched capacity. Establishes whether message passing contributes at all.
2. **Permuted graph.** Degree-preserving rewiring, so the graph has the same statistics and none of the biology. This separates “the graph carries signal” from “graph regularisation stabilises training”, and those are very different claims.

3. **Random- k sensitivity.** Sweep k (or r) over an order of magnitude and report the curve, not the best point. A method whose ARI moves by 0.15 between $k = 10$ and $k = 20$ has a hyperparameter, not a result.
4. **Trivial baseline.** SGC (§B.7) or PCA + Leiden, with resolution tuned exactly as the method’s own clustering was. This is the baseline most often missing and most often competitive.

E.3 Leakage, in the forms it takes here

Transductive graph leakage. In semi-supervised node classification the whole graph — including test nodes’ features — is used to build $\hat{A}$ and to compute representations. This is legitimate in the transductive setting *if the paper says so*, and it makes the numbers incomparable to inductive methods, which must handle unseen cells.

Preprocessing outside the fold. HVG selection, PCA, and graph construction on the full dataset before cross-validation leaks test information into every fold. In [RL §7.1], where cohorts are small, this alone can account for several points of accuracy.

Reference leakage in deconvolution and label transfer. Simulating pseudo-spots from the reference used to score, or evaluating annotation on a query drawn from the same study as the reference.

Edge leakage in link prediction. Random edge splits when the underlying network has strong degree structure (§D.6).

E.4 A training recipe that usually works

Nothing here is novel; it is the set of defaults that saves the most time.

- **Preprocessing.** Library-size normalise to a fixed total, $\log_{1p}$ for anything that is not a count-likelihood model, 2000–3000 HVGs by Seurat v3/pearson_residuals flavour, and keep the raw counts for the likelihood head.
- **Architecture.** Two message-passing layers, hidden width 128–256, latent 10–32. Deeper needs §B.6 machinery, and on a k NN cell graph rarely pays.
- **Regularisation.** Dropout 0.2–0.5 on hidden units, DropEdge 0.1–0.2 if going beyond two layers, weight decay 10^{-4} – 10^{-5} .
- **Optimisation.** Adam at 10^{-3} , cosine or plateau decay, early stopping on a held-out reconstruction or clustering criterion — never on the metric being reported.
- **Warm-up.** Anneal the KL over the first 10–20% of training (§C.2); anneal the clustering loss in too, since DEC on an untrained embedding cements noise.
- **Seeds.** Five seeds minimum, and report mean and spread. In this literature the seed-to-seed spread of ARI on a single dataset is often as large as the gap between the top five methods, and a paper reporting one number per dataset has reported a draw from a distribution.
- **Sanity checks.** Confirm the graph’s connected components (§A.4); confirm the KL is not zero; confirm the reconstruction improves; look at the embedding before believing the score.

E.5 Software worth standardising on

- **PyTorch Geometric** and **DGL** — the two general graph libraries; PyG’s `MessagePassing` base class is the cleanest expression of Equation (6) in code, and its sampler covers §B.7.
- **scvi-tools** — reference implementations of the count VAEs of §C.1–C.2, and the right place to start a new generative model rather than reimplementing an NB likelihood.
- **DANCE** — a benchmark library covering many of the single-cell tasks in [RL §2], with standardised splits; the most direct way to obtain the trivial baselines of §E.2.
- **Squidpy** / **scanpy** — spatial graph construction, neighbourhood enrichment, Moran’s I ; use their graph builders rather than writing your own, so that “radius = 50 μm ” means the same thing across papers.
- **scIB** — the integration metric suite of §E.1.

Part F — The same ideas, in code

Equations hide the parts that break. This section gives minimal, dependency-light implementations of the pieces that recur, in the order they appear in a pipeline. Nothing here is optimised; everything here is meant to be read.

F.1 Building the graph

```
import numpy as np, scanpy as sc
from scipy.sparse import csr_matrix

def cell_knn_graph(adata, n_pcs=50, k=15, use_rep=None, mutual=False):
    """Standard kNN cell graph. Returns a symmetric sparse adjacency."""
    if use_rep is None:
        sc.pp.pca(adata, n_comps=n_pcs)
        use_rep = "X_pca"
    sc.pp.neighbors(adata, n_neighbors=k, use_rep=use_rep)
    A = adata.obsp["connectivities"].copy() # UMAP fuzzy weights
    A = A.maximum(A.T) if not mutual else A.minimum(A.T)
    A.setdiag(0.0); A.eliminate_zeros()
    return csr_matrix(A)

def spatial_graph(coords, mode="radius", r=None, k=6):
    """Spatial neighbour graph from coordinates (n, 2) in physical units."""
    from sklearn.neighbors import radius_neighbors_graph, kneighbors_graph
    if mode == "radius":
        A = radius_neighbors_graph(coords, radius=r, mode="connectivity")
    else:
        A = kneighbors_graph(coords, n_neighbors=k, mode="connectivity")
    A = A.maximum(A.T) # undirected
    A.setdiag(0.0); A.eliminate_zeros()
    return csr_matrix(A)

def normalise(A, mode="sym"):
    """Propagation matrix:  $A_{\text{hat}} = D^{-1/2} (A + I) D^{-1/2}$ , or row-stochastic."""
    import scipy.sparse as sp
    n = A.shape[0]
    At = A + sp.eye(n, format="csr")
    deg = np.asarray(At.sum(1)).ravel()
    if mode == "sym":
        dinv = sp.diags(np.power(deg, -0.5, where=deg > 0))
        return (dinv @ At @ dinv).tocsr()
    dinv = sp.diags(np.divide(1.0, deg, where=deg > 0))
```

```
return (dinv @ At).tocsr()
```

Two details that cause real bugs. `setdiag(0)` before normalising, then adding I inside `normalise`, guarantees exactly one self-loop rather than two (scanpy's connectivities sometimes carries a diagonal). And `maximum(A.T)` is the union symmetrisation of §A.2.1 — swapping it for `minimum` gives mutual k NN, which is a one-character change with large downstream consequences.

F.2 The diagnostics of §A.4

```
def graph_report(A, labels=None):
    import scipy.sparse as sp
    deg = np.asarray((A > 0).sum(1)).ravel()
    n_comp, comp = sp.csgraph.connected_components(A, directed=False)
    out = {"n_nodes": A.shape[0], "n_edges": int((A > 0).nnz / 2),
          "deg_mean": deg.mean(), "deg_p99": np.percentile(deg, 99),
          "n_isolated": int((deg == 0).sum()), "n_components": n_comp,
          "largest_component_frac": np.bincount(comp).max() / A.shape[0]}
    if labels is not None:
        src, dst = A.nonzero()
        out["edge_purity"] = float((labels[src] == labels[dst]).mean())
    return out

def permute_graph(A, seed=0):
    """Degree-preserving rewiring: the ablation of §E.2, item 2."""
    import networkx as nx
    G = nx.from_scipy_sparse_array(A)
    nx.double_edge_swap(G, nswap=10 * G.number_of_edges(),
                       max_tries=100 * G.number_of_edges(), seed=seed)
    return nx.to_scipy_sparse_array(G, format="csr")
```

If `graph_report` shows `largest_component_frac` below ~ 0.95 , stop and fix the graph before training anything.

F.3 A message-passing layer from scratch

The matrix form of Equation (7) is four lines. Writing it once, rather than importing it, makes the shapes concrete:

```
import torch, torch.nn as nn, torch.nn.functional as F

class GCNLayer(nn.Module):
    def __init__(self, d_in, d_out, dropout=0.2):
        super().__init__()
        self.lin = nn.Linear(d_in, d_out, bias=True)
        self.drop = nn.Dropout(dropout)
```

```

        nn.init.xavier_uniform_(self.lin.weight)    # Glorot, as in the paper

    def forward(self, H, A_hat):                    # H: (n, d_in), A_hat sparse (n, n)
        H = self.drop(H)
        H = self.lin(H)                             # (n, d_out)
        return torch.sparse.mm(A_hat, H)           # (n, d_out)

class GCN(nn.Module):
    def __init__(self, d_in, d_hid, d_out):
        super().__init__()
        self.l1, self.l2 = GCNLayer(d_in, d_hid), GCNLayer(d_hid, d_out)

    def forward(self, X, A_hat):
        return self.l2(F.relu(self.l1(X, A_hat)), A_hat)

```

Attention (Equation 9) is where the memory goes, because coefficients are per edge, not per node. The edge-wise form, using `edge_index` of shape $(2, |E|)$:

```

class GATLayer(nn.Module):
    def __init__(self, d_in, d_out, heads=4, v2=True):
        super().__init__()
        self.h, self.d, self.v2 = heads, d_out, v2
        self.W = nn.Linear(d_in, heads * d_out, bias=False)
        self.a = nn.Parameter(torch.empty(heads, 2 * d_out))
        nn.init.xavier_uniform_(self.a)

    def forward(self, H, edge_index):
        src, dst = edge_index                       # message src -> dst
        Wh = self.W(H).view(-1, self.h, self.d)    # (n, heads, d)
        pair = torch.cat([Wh[dst], Wh[src]], dim=-1) # (|E|, heads, 2d)
        if self.v2:                                  # GATv2: nonlinearity first
            e = (F.leaky_relu(pair, 0.2) * self.a).sum(-1)
        else:                                         # GAT: projection first
            e = F.leaky_relu((pair * self.a).sum(-1), 0.2) # (|E|, heads)
        e = e - e.max()                              # softmax stability
        num = e.exp()
        den = torch.zeros(H.size(0), self.h, device=H.device)
        den = den.index_add_(0, dst, num) + 1e-16
        alpha = num / den[dst]                       # (|E|, heads)
        msg = alpha.unsqueeze(-1) * Wh[src]          # (|E|, heads, d)
        out = torch.zeros_like(Wh).index_add_(0, dst, msg)
        return out.reshape(H.size(0), self.h * self.d)

```

The `index_add_` pattern is the scatter-sum that PyG's `MessagePassing` wraps; seeing it explicitly makes clear why a GAT layer costs $O(|\mathcal{E}| \text{ heads } d)$ memory and a GCN layer does not.

F.4 The count likelihood

The ZINB negative log-likelihood of Equations (23)–(24), written stably. `pi_logits` is the raw logit for the zero-inflation gate, `theta` a per-gene inverse dispersion, `mu` the mean $\ell_c \rho_{cg}$:

```
def zinb_nll(x, mu, theta, pi_logits, eps=1e-8):
    """x, mu, pi_logits: (n, g); theta: (g,) or (n, g). Returns mean NLL."""
    theta = theta.clamp(min=eps)
    mu = mu.clamp(min=eps)
    log_theta_mu = torch.log(theta + mu + eps)

    # log NB(x | mu, theta)
    log_nb = (theta * (torch.log(theta + eps) - log_theta_mu)
              + x * (torch.log(mu + eps) - log_theta_mu)
              + torch.lgamma(x + theta) - torch.lgamma(theta) - torch.lgamma(x + 1))

    # gate probability is pi = sigmoid(pi_logits)
    case_zero = F.softplus(log_nb - pi_logits) - F.softplus(-pi_logits)
    case_nonzero = log_nb - F.softplus(pi_logits)
    log_prob = torch.where(x < eps, case_zero, case_nonzero)
    return -log_prob.sum(-1).mean()
```

The `torch.where` on `x < eps` is the whole of the zero-inflation logic: at zero the likelihood is the mixture $\pi + (1 - \pi)p_{\text{NB}}(0)$, and in log-space that is `softplus(log  $p_{\text{NB}}(0) - g$ ) - softplus(-g)` for gate logit g , which avoids the catastrophic cancellation of computing the mixture directly; away from zero it is `log  $p_{\text{NB}}(x) - \text{softplus}(g)$` , since `log(1 -  $\pi$ ) = -softplus(g)`. Note the asymmetry — `softplus(-g)` in one branch, `softplus(+g)` in the other. Getting that sign wrong is the most common bug in hand-rolled ZINB heads, and it is silent: the loss still decreases, but the likelihood no longer integrates to one, so the fitted dispersion drifts. The check is one line — `sum exp` of the log-probability over $x = 0, 1, 2, \dots$ for fixed parameters and confirm it reaches 1.

Dropping the `lgamma(x+1)` term is common (it is constant in the parameters) and changes the reported loss value but not the gradients — a frequent source of confusion when comparing loss curves across implementations.

F.5 Propagation without parameters

APPNP (Equation 14) is the cheapest fix for depth, and it is six lines:

```
class APPNPProp(nn.Module):
    def __init__(self, K=10, alpha=0.15):
        super().__init__()
        self.K, self.alpha = K, alpha

    def forward(self, H, A_hat):
        Z = H
        for _ in range(self.K):
```

```

        Z = (1 - self.alpha) * torch.sparse.mm(A_hat, Z) + self.alpha * H
    return Z

```

Used as $Z = \text{APPNPProp}()(\text{mlp}(X), A_hat)$, this gives a 10-hop receptive field with the parameter count of an MLP and none of the collapse in Figure 5.

F.6 A training loop with the checks built in

```

def train(model, X, A_hat, counts, lib, epochs=400, beta_max=1.0, warmup=60):
    opt = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-5)
    best, patience, wait = float("inf"), 40, 0
    for ep in range(epochs):
        model.train(); opt.zero_grad()
        z, mu_q, logvar_q, mu_x, theta, pi = model(X, A_hat)
        beta = beta_max * min(1.0, ep / warmup)  # KL warm-up, §C.2
        kl = -0.5 * (1 + logvar_q - mu_q.pow(2) - logvar_q.exp()).sum(-1).mean()
        loss = zinb_nll(counts, lib * mu_x, theta, pi) + beta * kl
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)
        opt.step()

        if ep % 20 == 0:  # the sanity checks
            print(f"ep {ep:4d}  loss {loss.item():.3f}  kl {kl.item():.4f}")
            assert kl.item() > 1e-3, "posterior collapse: latent is unused"
        if loss.item() < best - 1e-4:
            best, wait = loss.item(), 0
        else:
            wait += 1
            if wait > patience:
                break
    return model

```

The assertion is the point. Posterior collapse (§C.2) is silent — the model trains, the loss goes down, the embedding is noise — and a one-line check at every logging step catches it in the first minute rather than after the clustering results look strange.

F.7 The ablation harness

Everything in §E.2, as a loop over graph variants:

```

def ablation_suite(build_model, X, A, counts, lib, seeds=range(5)):
    import scipy.sparse as sp
    variants = {
        "real": A,
        "permuted": permute_graph(A),  # §E.2 item 2
    }

```

```

    "empty":    sp.csr_matrix(A.shape),                # §E.2 item 1 (MLP)
    "random":   sp.random(*A.shape, density=A.nnz / A.shape[0] ** 2,
                          format="csr", data_rvs=np.ones),
}
results = {}
for name, Av in variants.items():
    A_hat = to_torch_sparse(normalise(Av))
    scores = []
    for s in seeds:
        torch.manual_seed(s)
        m = train(build_model(), X, A_hat, counts, lib)
        scores.append(evaluate(m, X, A_hat))          # ARI, NMI, whatever
    results[name] = (np.mean(scores), np.std(scores))
return results

```

If real does not beat permuted by more than the seed-to-seed standard deviation, the graph is decoration. That is a result worth knowing before writing the paper, not after review.

F.8 Reading a benchmark table

A checklist for the tables that fill [RL §9], applied in the order that fails fastest:

1. **Was the preprocessing shared?** If each method ran its own pipeline, the table compares pipelines. Nearly all published comparisons fail this.
2. **How many seeds, and what is the spread?** One number per cell means the ranking is not identified.
3. **Was the clustering resolution tuned per method, and how?** Tuned against ground truth for the proposed method and left at default for baselines is the most common quiet distortion.
4. **Is the trivial baseline present?** PCA + Leiden, or SGC. Its absence is informative.
5. **Do the datasets favour the method's assumption?** A method with a spatial smoothing step, evaluated only on tissues with large contiguous domains, has been tested on its best case.
6. **Is the metric the one the use case needs?** ARI is not rare-population recovery; AUROC is not early precision.
7. **Who ran the comparison?** Self-reported tables and independent re-evaluations disagree systematically, and the reading list flags several specific cases where they do.

F.9 Ten things worth carrying away

1. There is no graph in the data. Every edge is an assumption, and the assumption is usually load-bearing.
2. k and depth are the same knob seen twice; a two-layer model on a $k = 30$ graph sees roughly 900 cells.
3. Two propagation steps help; thirty destroy. The turning point is measurable on your own graph (Figure 5).

4. If depth is genuinely wanted, decouple it from parameters with personalised PageRank rather than stacking layers.
5. Attention is a better default than fixed weights wherever boundaries matter, and GATv2 is strictly better than GAT at the same cost.
6. Count likelihoods matter more than architecture; MSE on log-counts quietly costs more than a layer type ever will.
7. Batch effects enter the graph before they enter the model, which is the strongest argument for graphs built from features or coordinates rather than from cells.
8. Fusion at the label level (VCDN) survives small n ; fusion at the feature level usually does not.
9. Every explanation — attention map, saliency, learned subgraph — is a property of one trained model until it is shown to be stable across seeds.
10. The permuted-graph ablation is the cheapest experiment in this field and the one most likely to change your conclusion.

Appendix

A. Complexity at a glance

Operation	Time	Memory	Notes
Dense layer	$O(nd^2)$	$O(nd)$	
GCN layer, full batch	$O(\mathcal{E} d + nd^2)$	$O(nd)$	$ \mathcal{E} = nk$ for k NN
GAT layer, K heads	$O(K(\mathcal{E} d + nd^2))$	$O(K(nd + \mathcal{E}))$	attention stored per edge
Full self-attention	$O(n^2d)$	$O(n^2)$	infeasible past $\sim 10^4$ nodes
k NN graph construction	$O(n \log n d)$ approx.	$O(nk)$	exact is $O(n^2d)$
Neighbour sampling, per batch	$O(Bs^L d)$	$O(Bs^L d)$	independent of n
APPNP, T steps	$O(T \mathcal{E} d)$	$O(nd)$	no extra parameters
METIS partition	$O(\mathcal{E})$ approx.	$O(n)$	one-off preprocessing

B. Identities used above

Laplacian spectrum. $L = I - \hat{A}$ is PSD with $\lambda \in [0, 2]$; $\lambda_1 = 0$ with eigenvector $\tilde{D}^{1/2}\mathbf{1}$; the multiplicity of 0 equals the number of connected components; $\lambda_n = 2$ if and only if a component is bipartite.

Dirichlet energy and the spectrum. $E(H) = \text{tr}(H^\top L H) = \sum_i \lambda_i \|\langle H, u_i \rangle\|^2$, so propagation by $\hat{A} = I - L$ multiplies the i -th component by $(1 - \lambda_i)$ and energy decays as $\max_{i \geq 2} (1 - \lambda_i)^{2L}$.

NB as gamma-Poisson. Marginalising $\lambda \sim \text{Gamma}(\theta, \theta/\mu)$ out of $\text{Poisson}(\lambda)$ gives (23); hence $\mathbb{E}[x] = \mu$, $\text{Var}[x] = \mu + \mu^2/\theta$.

Gaussian KL. For $q = \mathcal{N}(\mu, \text{diag } \sigma^2)$ and $p = \mathcal{N}(0, I)$, Equation (27).

Softmax Jacobian. $\partial \alpha_i / \partial e_j = \alpha_i (\delta_{ij} - \alpha_j)$ — the reason attention coefficients saturate and stop learning once one neighbour dominates.

C. Companion $\leftrightarrow$ reading list index

Reading list section	Companion sections
§1 Integration without graphs	C.1, C.2, C.4 (the baselines these methods define)
§2 Graph learning on single cells	A.2.1, B.1–B.4, C.3, C.5, D.1
§3 Graph-based multi-omic integration	A.2.3, B.5, C.4, D.2

Reading list section	Companion sections
§4 Spatial omics	A.2.2, B.6, D.3
§4.5 Spatial multi-omics	D.4
§5 Cell–cell communication	A.2.2, D.5
§6 Gene regulatory networks	C.3, D.6
§7.1 Patient-similarity networks	A.2.4, B.8, D.7
§7.2 Driver genes	B.10, D.7
§7.3–7.4 Survival, drug response	D.7
§8 Foundation models	D.8
§9 Reviews and benchmarks	E.1–E.3
§II.1–II.3 Origins, embeddings, convolution	B.1–B.3
§II.4 Attention and transformers	B.4
§II.5 Heterogeneous graphs	B.5
§II.6 Scalability	B.7
§II.7 Depth and over-smoothing	B.6
§II.8 Pooling	B.8
§II.9 Expressivity	B.3
§II.10 Self-supervised learning	B.9
§II.12 Explainability	B.10
§II.11 Dynamic and spatio-temporal graphs	B.11
§II.14 Libraries and benchmarks	E.5
§2.3 Imputation	D.1.1
§2.4 Trajectories and velocity	B.11, D.1.2
§2.5 Higher-order graphs	B.11

D. Figure credits

All figures were generated for this companion with Matplotlib; the scripts are in `figures-src/`. Figure 5 is computed, not drawn: it propagates random features on a 360-node stochastic block model with three communities ($p_{\text{in}} = 0.06$, $p_{\text{out}} = 0.006$) and reports Dirichlet energy and between/within cluster distance at each step, for plain propagation, a residual variant with $\beta = 0.5$, and APPNP with $\alpha = 0.15$. Re-running `fig04_oversmoothing.py` reproduces it exactly.